

课程须知

➤ 本课程适用软件版本：MWORKS.Syslab2023b 1030

➤ 本课程示例运行需要软件首选项加载：

基础库(TyBase)

数学库(TyMath)

图形库(TyPlot)

统计库(TyStatistics)

曲线拟合库(TyCurveFitting)

MWORKS.Syslab 曲线拟合工具箱应用

何一航

苏州同元软控信息技术有限公司

2025年5月23日



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

目录

1. 曲线拟合功能概述

2. 曲线拟合的数学模型

3. 曲线拟合及其实例

4. 曲面拟合及其实例

5. 曲线曲面的平滑以及插值

6. 关于样条及其构造

7. 样条的拟合平滑以及插值

1. 曲线拟合功能概述

现实问题

- 已知模型方程，参数未知
- 数据处理以及预测

实验

- 符合模型的数据
- 不要求模型曲线经过所有数据点
- 数据预处理

拟合

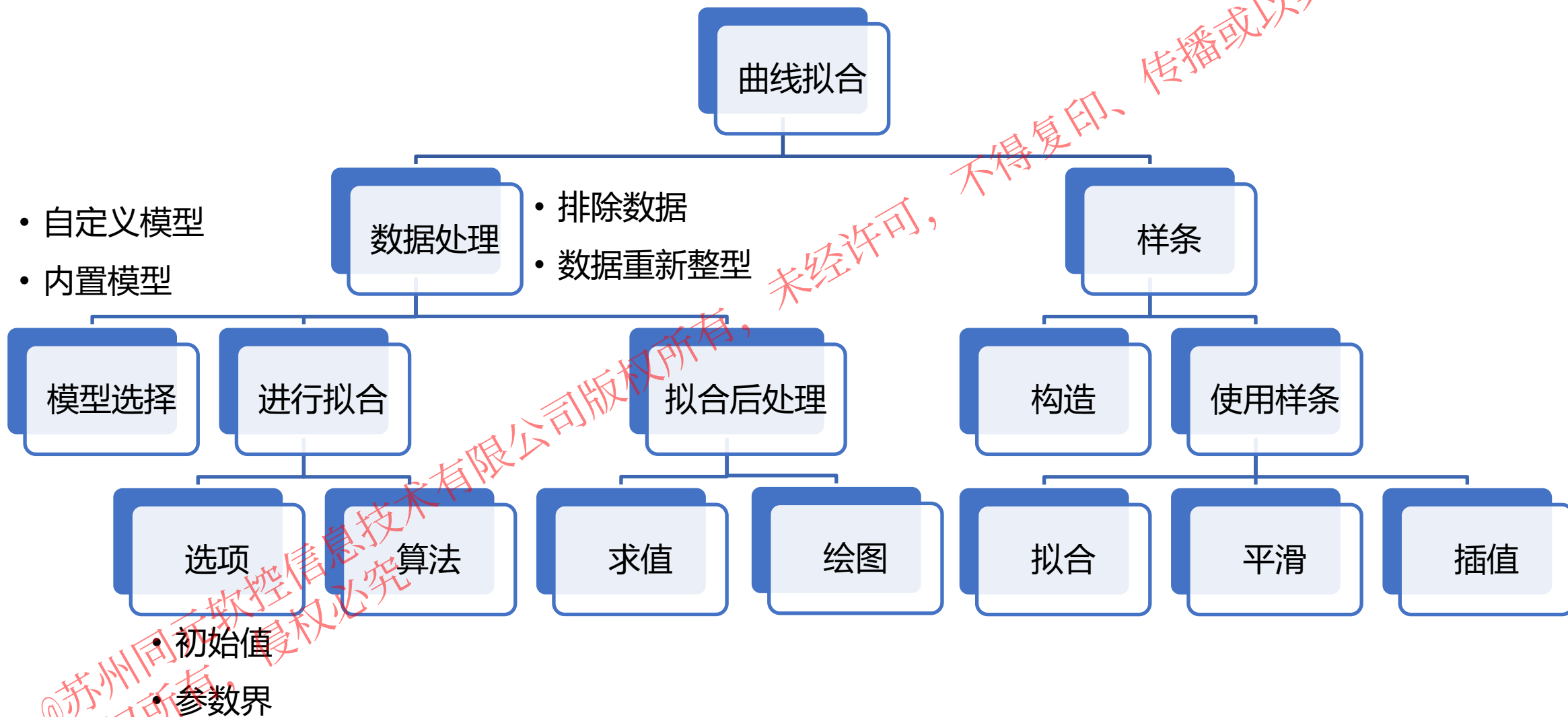
- 选择模型方程
- 设置初始值（预测值）

预测

- 预测基点位于原观测点上（数据处理）
- 预测基点位于原观测区间外

曲线拟合

1. 曲线拟合工具箱功能概述



目录

1. 曲线拟合功能概述

2. 曲线拟合的数学模型

3. 曲线拟合及其实例

4. 曲面拟合及其实例

5. 曲线曲面的平滑以及插值

6. 关于样条及其构造

7. 样条的拟合平滑以及插值

2.1 曲线拟合的数学模型 – 内置模型

模型名称	阶数	指定方法	方程
多项式	n	“polyn” 或 [“poly”, n]	$y = p_1x^n + \cdots + p_nx + p_{n+1}$
威布尔	无	“weibull”	$y = abx^{b-1}e^{-ax^b}$
指数	1	“exp1”	$y = ae^{bx}$
	2	“exp2”	$y = ae^{bx} + ce^{dx}$
傅里叶	n	“fouriern” 或 [“fourier”, n]	$y = a_0 + \cdots + a_n \cos(npx) + b_n \sin(npx)$
高斯	n	“gaussn” 或 [“gauss”, n]	$y = a_1e^{-\left(\frac{x-b_1}{c_1}\right)^2} + \cdots + a_ne^{-\left(\frac{x-b_n}{c_n}\right)^2}$
幂	1	“power1”	$y = ax^b$
	2	“power2”	$y = ax^b + c$
有理	m, n	“rationalmn”, m, n ≤ 9	$y = \frac{p_1x^m + \cdots + p_{m+1}}{x^n + q_1x^{n-1} + \cdots + q_n}$
正弦和	n	“sumsinen” 或 [“sumsine”, n]	$y = a_1 \sin(b_1x + c_1) + \cdots + a_n \sin(b_nx + c_n)$

曲线拟合的内置模型

2.1 曲线拟合的数学模型 – 内置模型

类别	模型名称	指定方法
插值	线性插值	“linearinterp”
	最邻近插值	“nearestinterp”
	保形分段三次 Hermite 插值	“pchipinterp”
	三次样条插值	“cubicinterp” ^[1]
样条	三次插值样条	“cubicspline”
	平滑样条	“smothingspline”

曲线拟合的其他内置模型

类别	模型名称	指定方法
拟合	多项式 ^[2]	“polymn”, m,n<=9
插值	线性插值	“linearinterp”
	最邻近插值	“nearestinterp”
Lowess	局部线性回归	Loess
	局部二次回归	Lowess
	局部三次回归	Locess

曲面拟合的内置模型

注1: cubicinterp 与 pchipinterp 相同

注2: 曲面拟合多项式为

$$y = p_{00} + p_{10}x + p_{01}y + \cdots + p_{ij}x^i y^j + \cdots + p_{m0}x^m + \cdots + p_{0n}y^n, i + j \leq \max(m, n)$$

2.1 曲线拟合的数学模型 – 自定义模型

除了工具箱的内置模型外，曲面拟合对于线性和非线性最小二乘同样支持用户的自定义模型。用户的自定义模型有三种输入方法，其说明和要求如下。

传入类型	描述	方程	类型	示例	示例方程
字符串	包含自变量与问题参数的可以运算的字符串。	<code>Meta.eval(func)</code>	非线性	<code>"a*sin(c*x)+b"</code>	$y(x) = a \sin(cx) + b$
字符串向量	每个成员均只能包含自变量且可运算，"1"是可以支持的成员。该用法不支持曲面拟合。	<code>sum(Meta.eval.(func).*t)</code>	线性	<code>"1"</code> <code>"sin(x)"</code> <code>"exp(x)"</code>	$y(x) = a + b \sin(x) + ce^x$
函数	以 <code>func(x,t,p)</code> 形式进行计算的函数，其中 <code>x</code> 为标量观测值或二元素向量观测值， <code>t</code> 为拟合参数， <code>p</code> 为问题参数（如果指定了问题参数）。	<code>func(x,t,p)</code>	非线性	<code>t[1]*sin(p*x)+t[2]</code>	$y(x) = a \sin(px) + b$

注：传入函数时最好能够保证函数的定义域是全平面，或者至少保证在参数上下界之间的所有可能值都合法，这样可以有效避免拟合由于函数无法计算而中途停止。

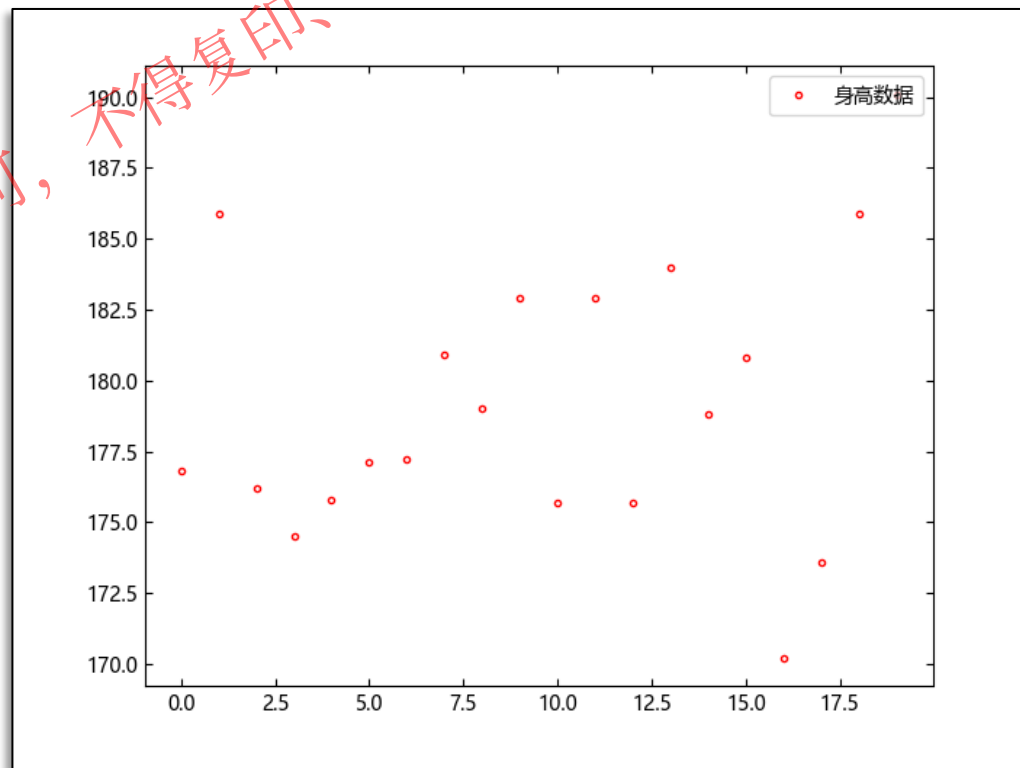


2.2 曲线拟合的数学模型 – 使用曲线拟合工具箱进行统计分析

对人群的身高进行抽样调查，得到 20 个人的身高数据分别为 176.8, 185.9, 176.2, 174.5, 175.8, 177.1, 177.2, 180.9, 179, 182.9, 175.7, 182.9, 175.7, 184, 178.8, 180.8, 170.2, 173.6, 185.9, 190.1 (单位 cm)。假设这些人的身高服从正态分布，我们希望能够估计这个正态分布的均值与标准差。

输入数据并绘制原数据点

```
x = [176.8, 185.9, 176.2, 174.5,  
175.8, 177.1, 177.2, 180.9, 179,  
182.9, 175.7, 182.9, 175.7, 184,  
178.8, 180.8, 170.2, 173.6, 185.9,  
190.1]  
plot(x, "r.")  
legend("身高数据", loc="northeast")
```



方法一：使用原数据的均值与标准差估计正态分布的均值与标准差

```
mu = mean(x) ## mu = 179.2  
sigma = sqrt(sum(t - mu,  
x) / (length(x) - 1)) ## sigma =  
4.911533151138947
```

这样我们估计的均值为 179.2，标准差为 4.9

抽样调查的身高数据

2.2 曲线拟合的数学模型 – 使用曲线拟合工具箱进行统计分析

方法二：使用概率密度函数

- 数据处理

- 我们把原数据分组（按组数均分），将组频率作为组概率，将组中值作为值，这样就可以得到值-概率数对

- 模型选择

- 原数据为正态分布，我们可以选择 gauss1 模型函数，也可以使用正态分布密度函数 normpdf

- 拟合参数

- Gauss1 模型的数学表达式为 $y = a_1 e^{-\left(\frac{x-b_1}{c_1}\right)^2}$ ，而正态分布的概率

密度函数为 $y = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$ ，因此最后的拟合参数为 $\mu = b_1$ ， $\sigma =$

$$\frac{c_1}{\sqrt{2}}$$

- 对于 normpdf，输出参数即为我们想要的参数。

```
## 构造以 nbins 为参数的函数，方便画图
fitdata = nbins -> begin
    freq, center = histcounts(x, nbins;
    nargout=2)
    ### 将数据以 nbins 组数平均分组，得到组频率
    freq 与组端点 center
    xdata = (center[1:(end - 1)] +
    center[2:end]) ./ 2
    ### 横坐标为组中值，即组端点的移动均值
    ydata = vec(freq) ./ 20
    ### 纵坐标为组概率，即组频率与总频率的商

    f1 = fit("gauss1", xdata, ydata)
    ### 使用 gauss1 模型进行拟合，该模型自带可用的
    初始值
    f2 = fit((x, t) -> normpdf(x, t[1], t[2]),
    xdata, ydata; startpt=[170, 20])

    ### 使用 normpdf 进行拟合，需要自定义初始值

    mu1 = f1.params[2]
    sigma1 = f1.params[3] / sqrt(2)
    mu2, sigma2 = f2.params
    return mu1, sigma1, mu2, sigma2
end
```

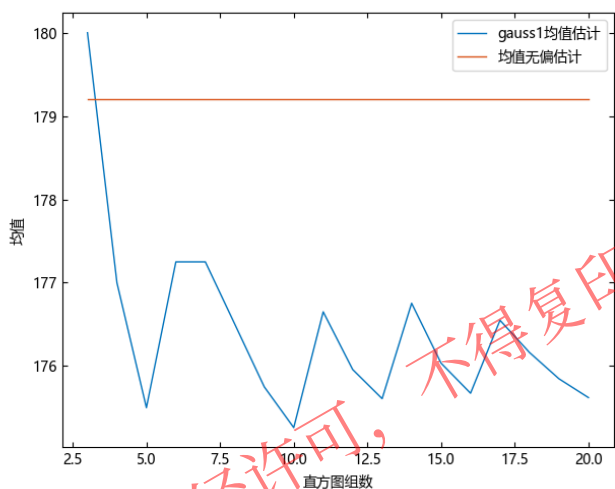
2.2 曲线拟合的数学模型 – 使用曲线拟合工具箱进行统计分析

方法二：使用概率密度函数

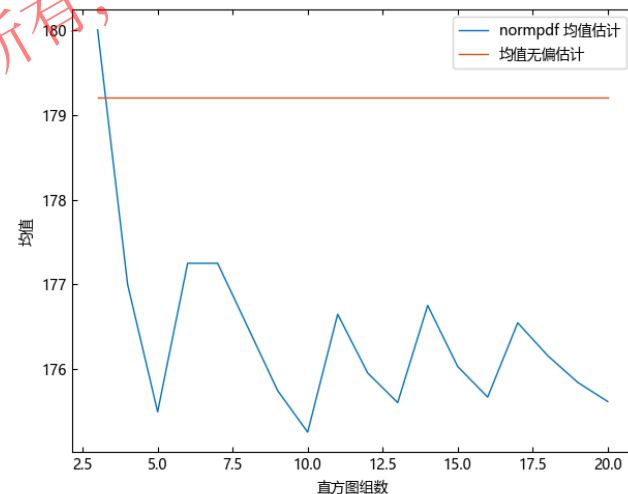
```
## 使用函数计算数据
trials = 3:20
results = fitdata(trials)
mu1 = getindex(results, 1)
sigma1 = getindex(results, 2)
mu2 = getindex(results, 3)
sigma2 = getindex(results, 4)

## 绘制图像
figure("使用gauss1模型的均值估计")
plot(trials, mu1)
xlabel("直方图组数")
ylabel("均值")
hold("on")
plot(trials, fill(mu,
length(trials)))
legend(["gauss1均值估计", "均值无偏估计"]; loc="northeast")
figure("使用gauss1模型的标准差估计")
plot(trials, sigma1)
xlabel("直方图组数")
ylabel("标准差")
hold("on")
plot(trials, fill(sigma,
length(trials)))
legend(["gauss1 标准差估计", "标准差的无偏估计"]; loc="northeast")
```

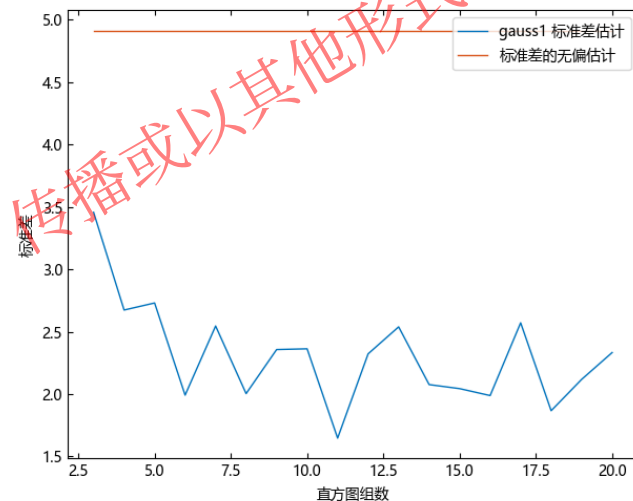
```
figure("使用 normpdf 模型的均值估计")
plot(trials, mu1)
xlabel("直方图组数")
ylabel("均值")
hold("on")
plot(trials, fill(mu,
length(trials)))
legend(["normpdf 均值估计", "均值无偏估计"]; loc="northeast")
figure("使用 normpdf 模型的标准差估计")
plot(trials, sigma1)
xlabel("直方图组数")
ylabel("标准差")
hold("on")
plot(trials, fill(sigma,
length(trials)))
legend(["normpdf 标准差估计", "标准差的无偏估计"]; loc="northeast")
```



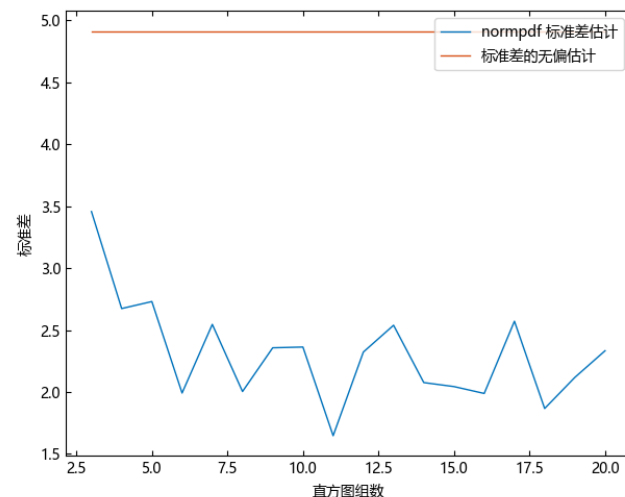
使用 gauss1 模型的均值估计



使用 normpdf 模型的均值估计



使用 gauss1 模型的标准差估计



使用 normpdf 模型的标准差估计

2.2 曲线拟合的数学模型 – 使用曲线拟合工具箱进行统计分析

方法三：使用概率分布函数

① 数据处理

- 概率分布函数表示了分布变量小于等于某个值的概率，这里需要拟合概率分布函数，就需要得到值-概率数对
- 我们将数据值排序，那么这个排序后的概率分布函数在第一个数据点的函数值估计就可以认为是 1，后面的数据可以以此类推

```
sx = sort(x)
sy = (1:20) ./ 20
```

② 模型选择

- normcdf 函数接受的参数就是正态分布的参数，因此拟合参数值就等于正态分布参数的估计值

```
f3 = fit((x, t) → normcdf(x, t[1], t[2]),
sx, sy, startpt=[190, 20])
### 使用 normcdf 进行拟合，需要自定义初始值
```

③ 拟合参数

- 原数据为正态分布，而正态分布的概率分布函数为 normcdf 函数

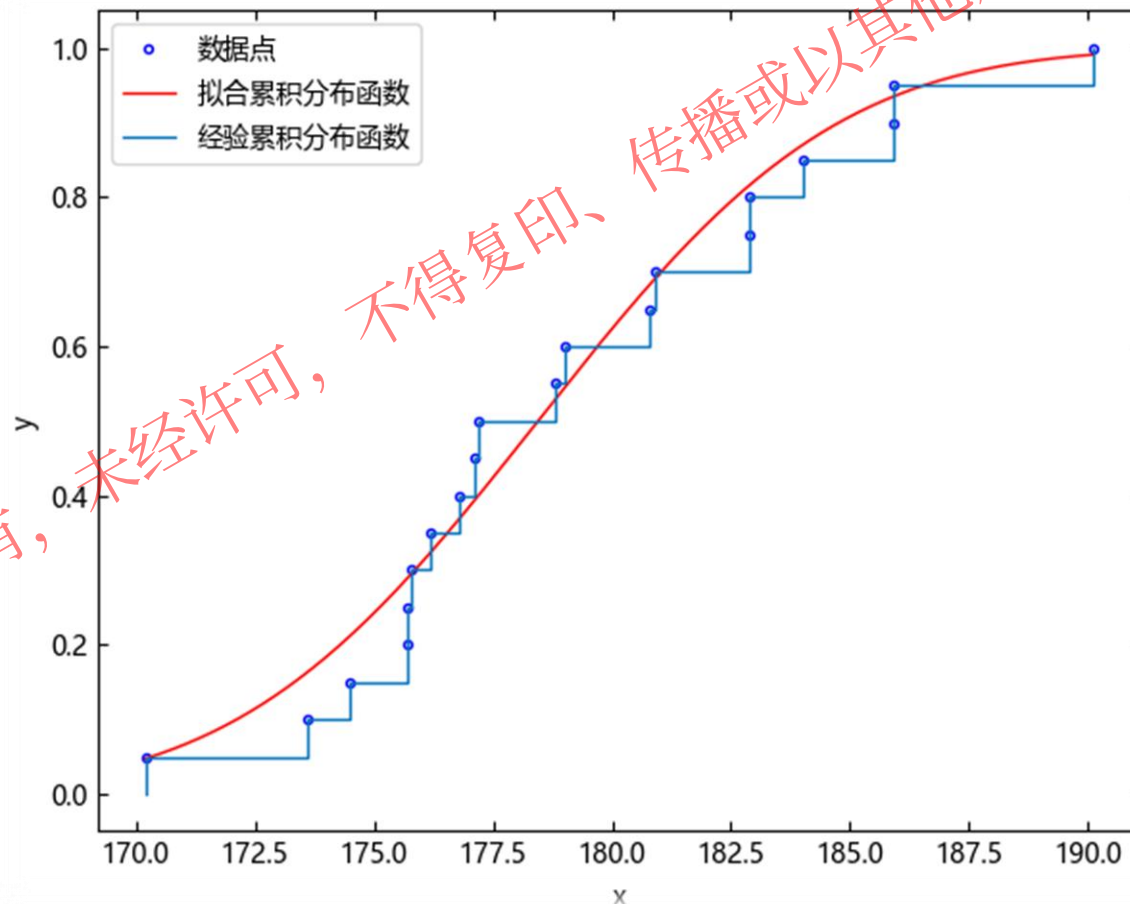
```
mu, sigma = f3.params

# 拟合参数与信赖区间 (0.95)
# t1 = 178.44790330214263
# t2 =
(178.07137772991953, 178.82442887436574)
4.933751036507835 (4.281870950571118,
5.585631122444552)
```

2.2 曲线拟合的数学模型 – 使用曲线拟合工具箱进行统计分析

方法三：使用概率分布函数

```
figure("使用 cdf 模型的估计")  
plotfit(f3, sx, sy)  
hold("on")  
ecdf(sx, plot=true)  
legend(["数据点", "拟合累积分布函数", "  
经验累积分布函数"], loc="northwest")
```



使用 normcdf 模型估计的累积分布函数

目录

1. 曲线拟合功能概述

2. 曲线拟合的数学模型

3. 曲线拟合及其实例

4. 曲面拟合及其实例

5. 曲线曲面的平滑以及插值

6. 关于样条及其构造

7. 样条的拟合平滑以及插值

3.1 曲线拟合及其实例— 曲线拟合

曲线拟合是构建曲线或数学函数的过程，该曲线或数学函数最接近一系列数据点，可能受到约束。曲线拟合可以涉及插值（需要精确拟合数据）或平滑，其中构建了一个近似拟合数据的“平滑”函数。拟合曲线可用作数据可视化的辅助手段，在没有可用数据的情况下推断函数值，并总结两个或多个变量之间的关系。外推法是指使用超出观测数据范围的拟合曲线，并且存在一定程度的不确定性，因为它可能反映了用于构建曲线的方法，也反映了观测数据。

数学地说，曲线拟合问题为给定观测点 $x[n]$ ，数据点 $y[n]$ ，参数曲线 $f(x; p): \mathbb{R} \times \mathbb{R}^m \rightarrow \mathbb{R}$ ，误差测度函数 $err(x): \mathbb{R}^n \rightarrow \mathbb{R}$ ，目标为找到 p_0 ，使得 $err(f(x, p))$ 最小。如果我们取 $err(x) = \sum (x - y)^2$ ，这成为最标准曲线拟合的最小二乘问题。

可以看出，曲线拟合问题是一个求函数最小值的问题，属于优化问题的一个子类，因此能够应用于优化问题的方法大多都能够应用于曲线拟合问题。

曲线拟合问题的常用算法有牛顿法（线搜索）和信赖域方法，曲线拟合工具箱所使用的所有方法都属于信赖域算法。

3.2 曲线拟合及其实例- 拟合算法与选项

目前曲线拟合工具箱进行曲线拟合时，允许用户对拟合选项进行自定义，这包括算法控制选项与数据处理选项，数据处理选项如下，更多选项请参考 fitoptions 函数帮助文档。

选项	数据类型	适用类型	含义
normalize	Bool	全部	中心化并缩放数据
exclude	指标向量或逻辑值向量		排除数据
weights	数值向量		拟合权重
robust	“LAR” “bisquare”	非线性最小二乘	稳健选项
lb,ub	数值向量		拟合参数的上下界
startpt	数值向量		拟合初始值

注：对于函数句柄类型的用户自定义模型，用户必须指定 startpt，可以将 startpt 指定为整数（表示函数参数个数，拟合选取该长度的随机数作为拟合初值，此时由于拟合使用初始向量为随机数，因此每次拟合结果都可能不一样）或者也可以直接指定 startpt 为参数个数长度的向量。



3.2 曲线拟合及其实例- 拟合算法与选项

曲线拟合工具箱目前对于线性和非线性拟合都具有多种算法可供选择，陈列如下。

对选项的要求	指定方法	描述
无	dogleg	狗腿法
	double dogleg	双狗腿法
	subspace 2d	子空间 2d 法
	cg steihaug	带 CG 的狗腿法
	levenberg marquardt	LM 算法
	fsubspace 2d	最小二乘子空间 2d 法
	trust region	信赖域反射法
参数无界	ty levenberg marquardt	带加速选项的 LM 算法

对于非线性拟合的算法

对选项的要求	指定方法	描述
无 参数无界	trust region	线性信赖域算法
	interior point	内点法
	mldivide	qr 分解

对于线性拟合的算法

拟合算法与选项既可以通过 fitoptions 函数指定，也可以在拟合时通过关键字参数指定。对于该算法不支持的参数通常可以传入，但是不会进行实际的使用。



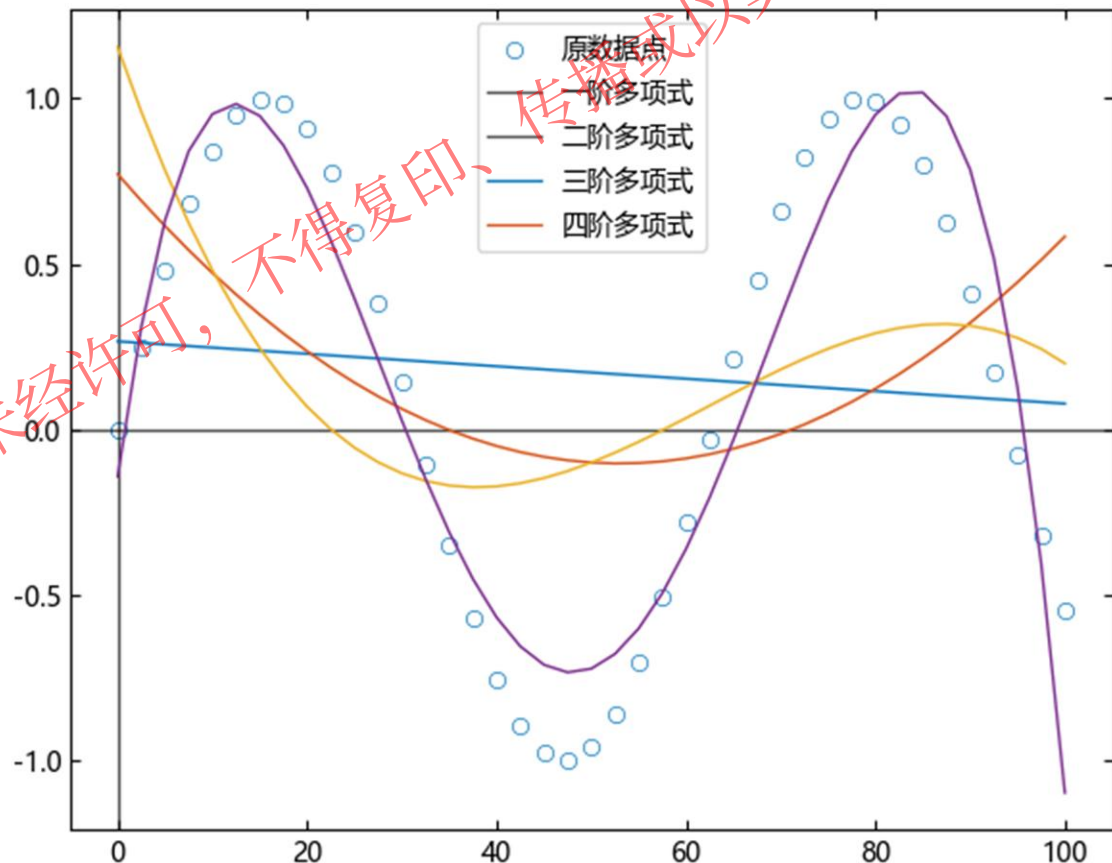
3.3 曲线拟合及其实例— 正弦数据的多项式拟合

使用 polyn 作为拟合模型可以方便地进行多项式拟合

```
xdata = LinRange(0,100,41)
ydata = sin.(0.1.*xdata)
### 生成正弦的测试数据

f1 = fit("poly1",xdata,ydata)
yfit1 = f1(xdata)
### 一阶多项式拟合与求值
f2 = fit("poly2",xdata,ydata)
yfit2 = f2(xdata)
### 二阶多项式拟合与求值
f3 = fit("poly3",xdata,ydata)
yfit3 = f3(xdata)
### 三阶多项式拟合与求值
f4 = fit("poly4",xdata,ydata)
yfit4 = f4(xdata)
### 四阶多项式拟合与求值

### 绘制原数据与拟合数据
figure("正弦数据的多项式拟合")
scatter(xdata,ydata)
hold("on")
xline(0)
yline(0)
plot(xdata,yfit1)
plot(xdata,yfit2)
plot(xdata,yfit3)
plot(xdata,yfit4)
legend(["原数据点","一阶多项式","二阶多项式","三阶多项式","四阶多项式"])
```



正弦数据的多项式拟合

3.4 曲线拟合及其实例- 非线性人口普查拟合

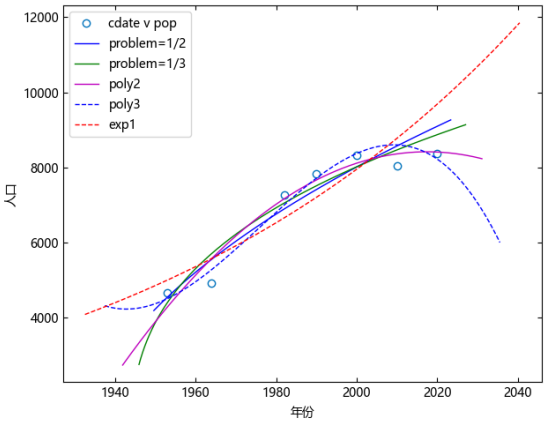
使用不同的模型对数据进行拟合

描述	方程
问题参数为 1/2 的自定义模型	$y = a(x - b)^{1/2}$
问题参数为 1/3 的自定义模型	$y = a(x - b)^{1/3}$
二阶多项式模型	$y = ax^2 + bx + c$
三阶多项式模型	$y = ax^3 + bx^2 + cx + d$
一阶指数模型	$y = ae^{bx}$

```
## 生成数据
cdate = [1953, 1964, 1982, 1990, 2000, 2010, 2020]
pop = [4667, 4912, 7265, 7835, 8329.1, 8041.8, 8367.5]
ft = fittype("a*(x-b)^n", problem="n")
opt = fitoptions(ft, lb=[0, 0], ub=[Inf, minimum(cdate)], startpt=[1, 1])
opt2 = fitoptions(ft, normalize=true)
curve2 = fit(ft, cdate, pop; problem=1 / 2, options=opt)
curve3 = fit(ft, cdate, pop; problem=1 / 3, options=opt)
population2 = fit("poly2", cdate, pop; options=opt2)
population3 = fit("poly3", cdate, pop; options=opt2)
populationExp = fit("exp1", cdate, pop; options=opt2)
```

将这些模型与原数据点绘制在一张图上

```
figure("各种拟合模型的对比")
plot(cdate, pop, "o")
hold("on")
xlabel("年份")
ylabel("人口")
plotfit(curve2, "b")
plotfit(curve3, "g")
plotfit(population2, "m")
plotfit(population3, "b--")
plotfit(populationExp, "r-")
hold("off")
legend(["cdate v pop", "problem=1/2", "problem=1/3", "poly2", "poly3", "exp1"]; loc="northwest")
```



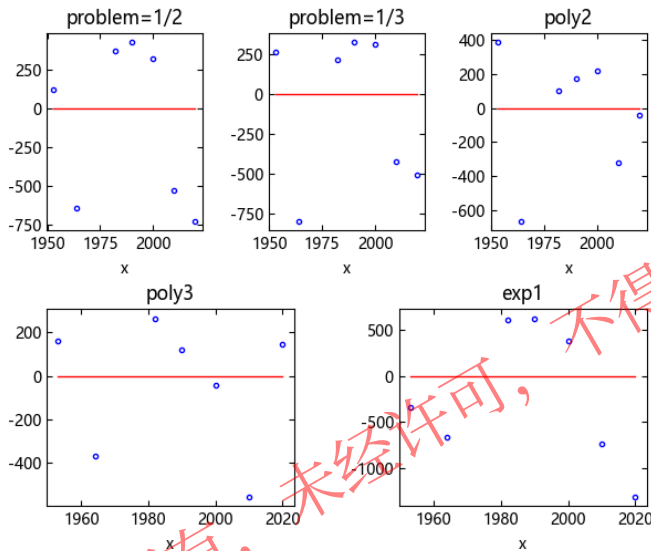
各种拟合模型的对比



3.4 曲线拟合及其实例- 非线性人口普查拟合

绘制这些模型的残差图像

```
figure("各种拟合模型的残差对比")
subplot(2,3,1)
plotfit(curve2, cdate, pop, "residuals")
title("problem=1/2")
legend("off")
subplot(2,3,2)
plotfit(curve3, cdate, pop, "residuals")
title("problem=1/3")
legend("off")
subplot(2,3,3)
plotfit(population2, cdate, pop, "residuals")
title("poly2")
legend("off")
tightlayout()
subplot(2,2,3)
plotfit(population3, cdate, pop, "residuals")
title("poly3")
legend("off")
subplot(2,2,4)
plotfit(populationExp, cdate, pop, "residuals")
title("exp1")
legend("off")
```



各种拟合模型的残差对比

poly3 模型具有最小的残差，但是外部图像变化过快，这表示使用 poly3 出现了过拟合现象，同样的现象也出现在了 problem=1/3 的自定义模型和 exp1 的内置模型

```
开始加载库，这可能需要一段时间...
加载完成。
[0x7F00753370] ANKALY: meaningless REX prefix used
[0x7F00753370] ANKALY: meaningless REX prefix used
[0x7F00753370] ANKALY: use of REX.w is meaningless (default operand size is 64)
[0x7F00753370] ANKALY: use of REX.w is meaningless (default operand size is 64)
[0x7F00753370] ANKALY: use of REX.w is meaningless (default operand size is 64)
[0x7F00753370] ANKALY: use of REX.w is meaningless (default operand size is 64)
ERROR: PyError (PyObject Escape, ((call(&= C:\Users\Public\Tongyuan\julia\dev\bin\julia.exe, PyPtr, (PyPtr, PyPtr, PyPtr), 0, pyargsptr, kw))))
<class 'AttributeError'>
AttributeError: module 'matplotlib.figure' has no attribute 'warn_external'
File "m_interface.py", line 2284, in adjust_view
File "C:\Users\Public\Tongyuan\julia\dev\bin\julia.exe\site-packages\matplotlib\figure.py", line 1192, in tight_layout
pad=pad, h_pad=h_pad, w_pad=w_pad, rect=rect)
File "m_figure.py", line 1722, in get_tight_layout_figure

Stacktrace:
[1] pyerr_check
# C:\Users\Public\Tongyuan\julia\dev\bin\julia.exe\julia\src\error.jl:25 [inlined]
[2] pyerr_check
# C:\Users\Public\Tongyuan\julia\dev\bin\julia.exe\julia\src\error.jl:25 [inlined]
[3] _handle_error(msg::String)
# C:\Users\Public\Tongyuan\julia\dev\bin\julia.exe\julia\src\error.jl:109 [inlined]
[4] macro expansion
# C:\Users\Public\Tongyuan\julia\dev\bin\julia.exe\julia\src\error.jl:109 [inlined]
[5] (::PyCall.var{#kw{::PyCall.PyObject, PyCall.PyObject, PyCall.PyObject}}{::PyCall.PyObject, PyCall.PyObject})()
# PyCall C:\Users\Public\Tongyuan\julia\dev\bin\julia.exe\src\pycall.jl:143 [inlined]
[6] disable_sigint
# C:\Users\Public\Tongyuan\julia\dev\bin\julia.exe\src\main.jl:107 [inlined]
```

注意：该版本会出现error为报错信息，提示用户图形该图形可能存在taylorlayout问题，后续版本20231230会消除该信息，不影响结果



3.4 曲线拟合及其实例- 非线性人口普查拟合

拟合比较 – problem = ½ 的自定义模型还是 poly2

查看这两个模型的统计优度，拟合模型的统计优度在其 s_data 域元

curve2.s_data

```
# R方: 0.899902520042962
# SSE: 1.5281986701031732e6
# DFE: 5.0
# 调整R方: 0.8798830240515544
# RMSE: 552.8469354356906
```

population2.s_data

```
# R方: 0.9510034705479256
# SSE: 722584.7279341616
# DFE: 4.0
# 调整R方: 0.9265052058218883
# RMSE: 425.02491925008394
```

查看这两个模型的置信区间，拟合模型的置信区间可以使用 confint 或者直接 display 来查看

```
confint(curve2)[1]
```

```
# 2x2 Matrix{Float64}:
#   760.675  1095.75
#   1908.54   1948.07
```

curve2

```
# General model:
# y(a,b,n,x) = a*(abs(x-b))^n
```

```
# 拟合参数与信赖区间 (0.95)
```

```
# a = 928.2132490403741 (760.6746432144903, 1095.7518548662579)
# b = 1928.3052476857752 (1908.5434235238297, 1948.0670718477206)
```

```
confint(population2)[1]
```

```
# 3x2 Matrix{Float64}:
#   -1170.73    14.0421
#    897.614   1887.88
#    6902.32   8253.99
```

population2

```
# Linear model poly2:
# y(p1,p2,p3,x) = p1*x^2 + p2*x + p3
```

```
#
# x is normalized by mean 1988.4285714285713 with std 24.123689206926475
#
```

```
# 拟合参数与信赖区间 (0.95)
```

```
# p1 = -578.3435091252754 (-1170.7291144307978, 14.042096180247086)
# p2 = 1392.7445330338262 (897.6137250386126, 1887.8753410290396)
# p3 = 7578.159604685163 (6902.324380270768, 8253.994829099558)
```

可以发现，poly2 模型的拟合效果更好。

目录

1. 曲线拟合功能概述

2. 曲线拟合的数学模型

3. 曲线拟合及其实例

4. 曲面拟合及其实例

5. 曲线曲面的平滑以及插值

6. 关于样条及其构造

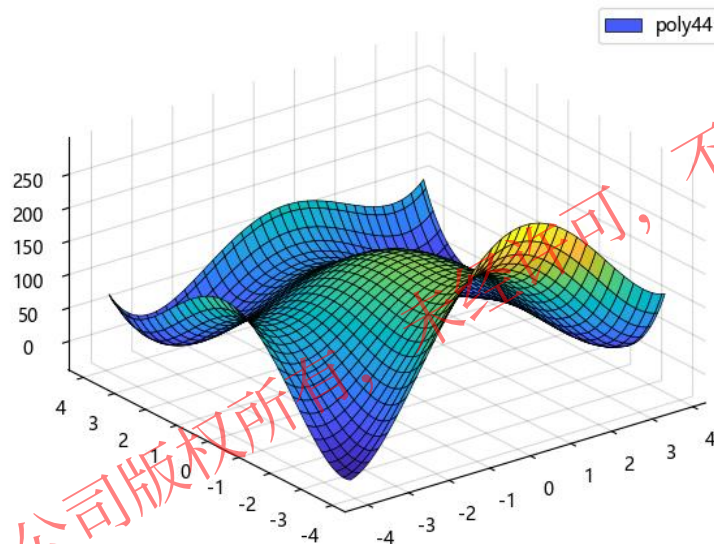
7. 样条的拟合平滑以及插值

4.1 曲面拟合及其实例- 多项式拟合

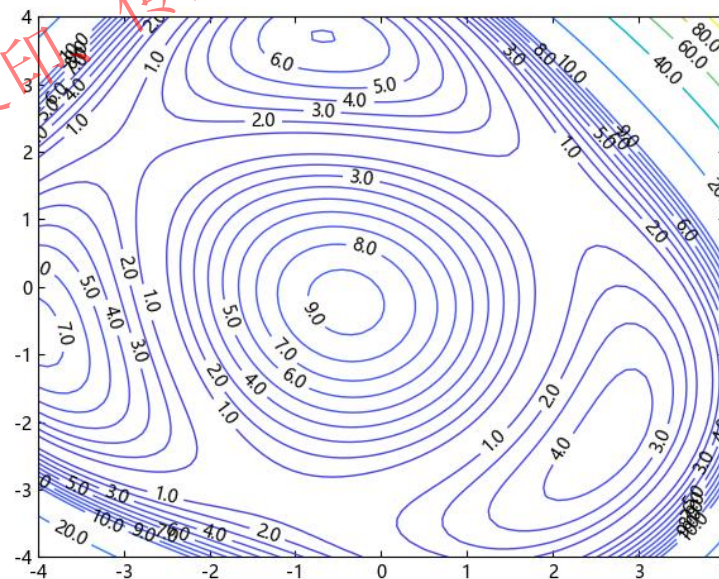
假设我们的数据来自于函数 $z = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$ ，为该数据增加 5% 的噪声，并用模型 poly44 进行拟合

```
fmodel = (x, y) -> (x^2 + y - 11)^2 + (x + y^2 - 7)^2
## 原模型
rng = MT19937ar(1)
## 固定随机数种子
num = 40
x = rand(rng, num) .* 8 .- 4
y = rand(rng, num) .* 8 .- 4
z = fmodel(x, y)
## 生成分布在 [-4,4] 的 40 个数据点
noise = randn(rng, num)
z = z .* (1 + 0.05 .* noise)
## 为数据点加入高斯噪声
ft = fit("poly44", [x y], z)
## 使用 poly44 模型拟合数据

figure("poly44 模型拟合")
## 绘制拟合图
plot3fit(ft)
legend(["poly44"])
figure("poly44 模型的残差等高线图")
## 绘制残差的等高线图
xdata, ydata = meshgrid2(LinRange(
4, 4, 100))
zdata = @. abs(fmodel(xdata, ydata)
- ft(xdata, ydata))
fcontour((x, y) -> abs(fmodel(x, y)
- ft(x, y)), [-4 4 -4 4],
levels=vcat(10:10, 20:20:100),
showtext="on")
```



poly44 模型拟合图



poly44 模型残差等高线图

4.2 曲面拟合及其实例- 自定义方程拟合

假设我们的数据来自于函数 $z = p + ax^b + cy^d + ex^by^d$ ，为该数据增加一定的噪声，并用该模型进行拟合。
正确的参数设置为 $a=1, b=2, c=1, d=2, e=1, p=0.5$

- 数据处理

- ① 定义以 $f(x,t,p)$ 形式调用的函数句柄。由于这次拟合为曲面拟合，因此 x 假设为二元素向量，而 t 为五元素向量，这样这个函数除了生成数据值外，还可以作为拟合模型。
- ② 固定随机数种子生成 x 和 y 数据，并依次生成 z 数据。对 z 数据增加噪声。

- 模型选择

- ① 使用数据处理时定义的函数作为本次拟合的模型。

```
func = (x,t,p)->p+t[1]*x[1]^t[2]+t[3]*x[2]^t[4]+t[5]*x[1]^t[2]*x[2]^t[4]
rng = MT19937ar(1)
num = 200
xdata = rand(rng,num)
ydata = rand(rng,num)
zdata = func.(eachslice([xdata ydata], dims=1), Ref([1, 2, 1, 2, 1]), 0.5) .* (0.5 .+ rand(rng, num))
```


4.2 曲面拟合及其实例- 自定义方程拟合

选项选择

- ① 由于模型中含有问题参数，因此需要指定问题参数的值，这一次尝试指定该值为 2。
- ② 由于模型为用户自定义的函数句柄类模型，因此必须指定 startpt 的值，这里指定 startpt 为长度为 5 的全 1 向量。

```
sfit = fit(func,[xdata ydata],zdata,startpt=ones(5),problem=2)
# General model:
# z(t1,t2,t3,t4,t5,p,x,y) = f(x,y,t,p)

# 拟合参数与信赖区间 (0.95)
# t1 = 3.9954093136361934, (0.39413860007269896, 7.596680027199688)
# t2 = 1.6312891748677973, (0.6882349511112229, 2.5743433986243716)
# t3 = -1.0111163222835837, (-1.1952209391206186, -0.8270117054465489)
# t4 = -0.09566346870822831, (-0.15827156533155035, -0.03305537208490629)
# t5 = -2.7207327369277805, (-6.131530305555744, 0.690064831700183)
```

具有矩形定义域的函数可能会在调用拟合函数 fit 时报错，这可以通过修改参数界来解决。具有非矩形定义域的函数最好能局限在矩形或者扩展至全平面以调用拟合函数（如果 fit 报错的话）。

模型处理

- ① 将拟合函数在范围 $[-2,2] \times [-2,2]$ 的矩阵中绘制。

```
plot3fit(sfit,"xlim",[-2,2],"ylim",[-2,2])
```

在调用 plot3fit 函数时报错了，这是由于拟合参数 t2 和 t4 中存在最简分式分子为奇数，分母为偶数的数，因此函数对于负值无法进行求值。

我们需要对模型进行修正，比如把 x 改为 abs(x) 以保持对称。

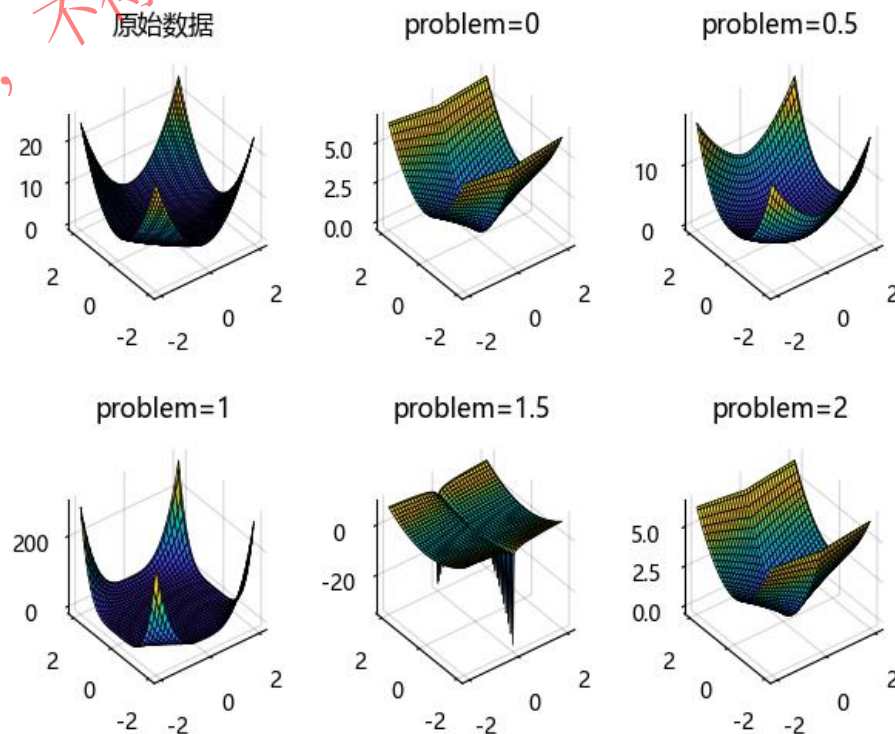
4.2 曲面拟合及其实例- 自定义方程拟合

对模型进行修正，将模型中的 x 全部改为 $\text{abs}(x)$ 并更改问题参数的值为 0: 0.5: 2 生成五个拟合。

将所有拟合绘制在一张图中。

```
figure("指定不同问题参数的曲面拟合")
subplot(2, 3, 1)
plotxdata = (LinRange(-2, 2, 100), LinRange(-2, 2, 100))
plotx, ploty = meshgrid2(plotxdata[1], plotxdata[2])
plotz = reshape([func([x, y], [1, 2, 1, 2, 1], 0.5) for x in plotxdata[1] for y in
plotxdata[2]], 100, 100)
surf(plotx, ploty, plotz)
hold("on")
title("原始数据")
subplot(2, 3, 2)
plot3fit(newfit1, "xlim", [-2, 2], "ylim", [-2, 2])
hold("on")
title("problem=0")
subplot(2, 3, 3)
plot3fit(newfit2, "xlim", [-2, 2], "ylim", [-2, 2])
hold("on")
title("problem=0.5")
subplot(2, 3, 4)
plot3fit(newfit3, "xlim", [-2, 2], "ylim", [-2, 2])
hold("on")
title("problem=1")
subplot(2, 3, 5)
plot3fit(newfit4, "xlim", [-2, 2], "ylim", [-2, 2])
hold("on")
title("problem=1.5")
subplot(2, 3, 6)
plot3fit(newfit1, "xlim", [-2, 2], "ylim", [-2, 2])
hold("on")
title("problem=2")
tightlayout()
```

```
newfunc = (x, t, p) -> func(abs.(x), t, p)
newfit1 = fit(newfunc, [xdata ydata], zdata, startpt=ones(5),
problem=0)
newfit2 = fit(newfunc, [xdata ydata], zdata, startpt=ones(5),
problem=0.5)
newfit3 = fit(newfunc, [xdata ydata], zdata, startpt=ones(5),
problem=1)
newfit4 = fit(newfunc, [xdata ydata], zdata, startpt=ones(5),
problem=1.5)
newfit5 = fit(newfunc, [xdata ydata], zdata, startpt=ones(5),
problem=5)
```



指定不同问题参数的曲面拟合

4.2 曲面拟合及其实例- 自定义方程拟合

从图中可以发现，最接近原始图像的似乎上是问题参数设置为 1 时的拟合而非问题参数设置为真实参数 0.5 时的，查看它们的拟合参数和信赖区间。（这可以通过将这些结构体直接打在命令行中看出，也可以通过 confint 函数）。

可以发现，实际上 newfit2 的拟合参数更接近真实值 [1,2,1,2,1] 而且它的信赖区间也一般地说更短。

在拟合结果中，也包含着它们的拟合统计优度，即是它们的 s_data 域元。将这些域元直接打在命令行可以直观看到它们的统计优度。

```
newfit2.s_data
# R方: 0.5986594309100783
# SSE: 33.941966014859204
# DFE: 195.0
# 调整R方: 0.5904268038518236
# RMSE: 0.4172066204876969
```

```
newfit3.s_data
# R方: 0.5028328724913866
# SSE: 42.04615990819923
# DFE: 195.0
# 调整R方: 0.4926345724399279
# RMSE: 0.464350441867519
```

```
newfit2
#= General model:
z(t1,t2,t3,t4,t5,p,x,y) = f(x,y,t,p)
```

拟合参数与信赖区间 (0.95)

```
t1 = 1.0237882088486678 (0.7910374006787954, 1.2565390170185402)
t2 = 1.7319337886473567 (1.0924120617707729, 2.3714555155239405)
t3 = 1.2582600967384436 (0.9915779847563542, 1.524942208720533)
t4 = 2.6133961710967193 (1.7860141910867602, 3.440778151106678)
t5 = 0.27586301613228414 (-0.30613111621165956, 0.8578571484762278)
=#
```

```
newfit3
#= General model:
z(t1,t2,t3,t4,t5,p,x,y) = f(x,y,t,p)
```

拟合参数与信赖区间 (0.95)

```
t1 = 0.6047788945015731 (0.3305017708005321, 0.879056018202614)
t2 = 3.197348923087246 (1.5287520622788933, 4.865945783895599)
t3 = 0.8474178869593453 (0.5330986089387124, 1.1617371649799781)
t4 = 4.405466672597319 (2.569772214501703, 6.241161130692935)
t5 = 1.3477870856263099 (0.4236003932257131, 2.271973778026907)
=#
```

在显示的信息中，一般来说，是 R 方、调整 R 方越接近 1 越好，SSE 和 RMSE 越小越好。在这些方面而言，都是 newfit2 更好。因此，我们可以说，newfit2 是最接近真实情况的拟合。

4.3 曲面拟合及其实例- 生物制药数据拟合

从文件中加载数据

使用 `fittype` 函数来定义模型，其中 CA 和 CB 是药物浓度，IC50A、IC50B、alpha 和 n 是要估计的系数

假设 $E_{max} = 1$ ，因为效果输出是正交化的

为稳健的拟合、边界和起点设置拟合选项。

```
using DelimitedFiles
data, header = readdlm(pkgdir(TyCurveFitting) *
"/examples/docs/OpioidHypnoticSynergy.txt"; header=true)
Propofol = data[:, 1]
Remifentanil = data[:, 2]
Laryngoscopy = data[:, 6]
```

```
ft = fittype(
    "Emax*(CA/IC50A + CB/IC50B +
alpha*(CA/IC50A)*(CB/IC50B))^n/((CA/IC50A+CB/IC50B+alpha*(CA/IC50A)*(CB/IC50B))^n+1)",
    independent=["CA", "CB"],
    dependent="z",
    problem="Emax",
)
```

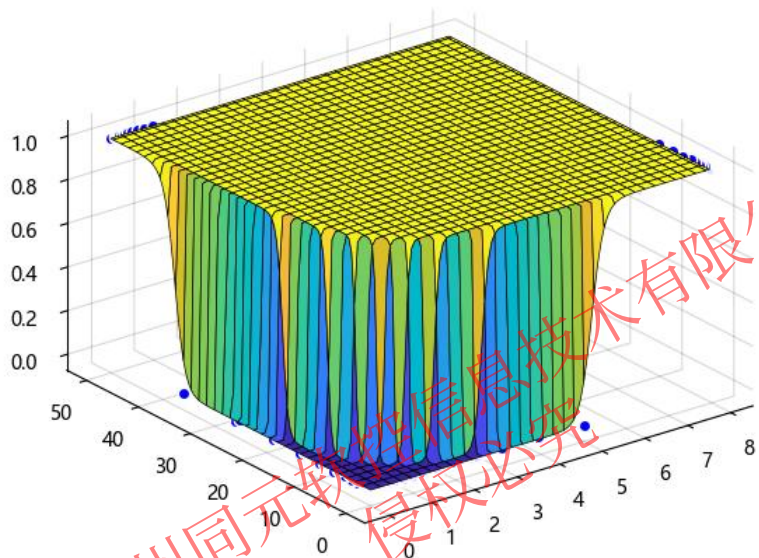
```
# General model:
# z(IC50A,IC50B,alpha,n,Emax,CA,CB) = Emax*(CA/IC50A + CB/IC50B +
alpha*(CA/IC50A)*(CB/IC50B))^n/((CA/IC50A+CB/IC50B+alpha*(CA/IC50A)*(CB/IC50B))^n+1)
```

```
Emax = 1
```

```
opts = fitoptions(ft)
opts.lb = [0, 0, -5, -0]
opts.robust = "LAR"
opts.startpt = [4, 4, 4, 4]
```

使用这些选项进行拟合并绘制。

```
f = fit(ft, [Propofol Remifentanil], Laryngoscopy, problem=Emax, options=opts)
plot3fit(f, [Propofol Remifentanil], Laryngoscopy)
```



生物制药数据拟合

目录

1. 曲线拟合功能概述

2. 曲线拟合的数学模型

3. 曲线拟合及其实例

4. 曲面拟合及其实例

5. 曲线曲面的平滑以及插值

6. 关于样条及其构造

7. 样条的拟合平滑以及插值

5.1 曲线曲面的平滑以及插值

除了拟合外，曲线拟合工具箱同时也集成了对给定的曲线曲面进行插值、平滑和局部回归的算法，fit 函数本身就集成了这些算法的基础部分，而对这些过程更细致的掌控就需要使用更相关的函数了。

类别	模型名称	指定方法	其他方法
插值	线性插值	“linearinterp”	interp1
	最邻近插值	“nearestinterp”	interp1
	保形分段三次 Hermite 插值	“pchipinterp”	interp1
	三次样条插值	“cubicinterp” ^[1]	interp1
样条	三次插值样条	“cubicspline”	spapi
	平滑样条	“smothingspline”	spaps

曲线数据支持的其他方法

模型名称	指定方法
局部线性回归	Loess
局部二次回归	Lowess
局部三次回归	Locess

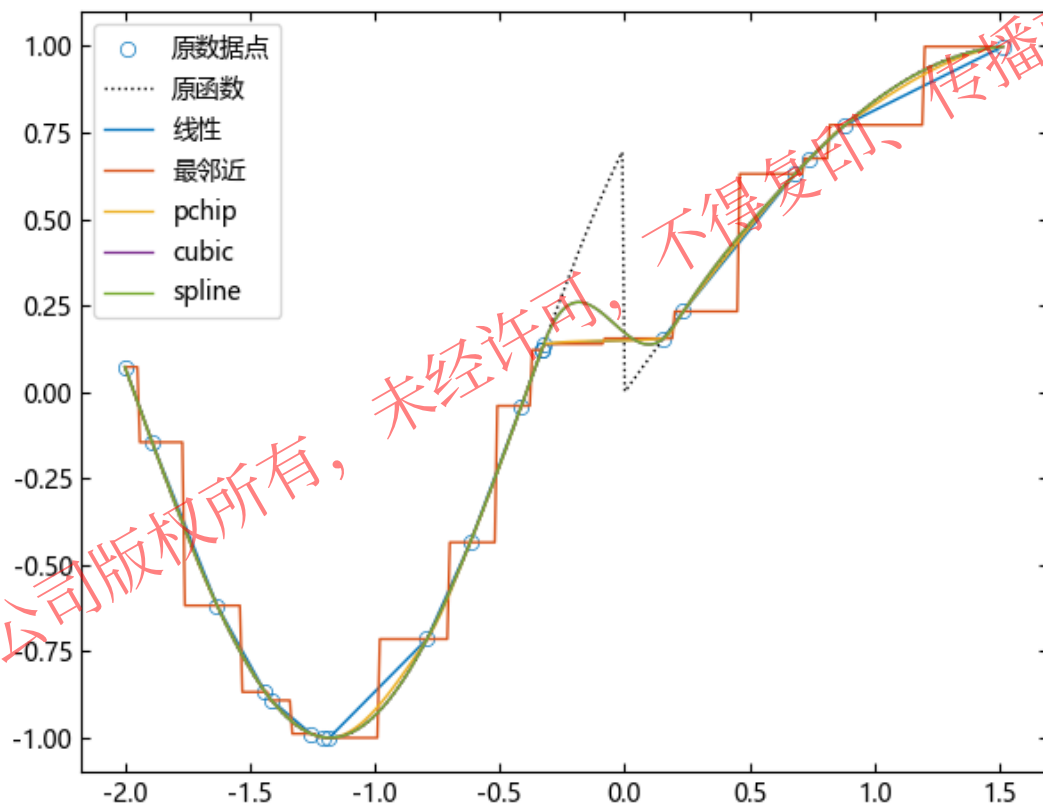
曲面数据支持的其他方法

对于平滑数据而言，除了使用 fit 函数的平滑样条方法以外，曲线拟合库中也使用传统的数据平滑方法的函数，smooth，这包括移动平均、局部回归等方法。

5.1 曲线曲面的平滑以及插值 – 对曲线数据进行插值

假设我们得到一组分段正弦函数的观测数据，我们希望对它进行各种不同方式的插值，并比较它们的插值结果。

```
fm = x -> x > 0 ? sin(x) : sin(2 *  
x + pi / 4)  
num = 20  
rng = MT19937ar(1)  
x = rand(rng, num) .* 4 .- 2  
y = fm.(x)  
linterp = fit("linearinterp", x, y)  
ninterp = fit("nearestinterp", x, y)  
pinterp = fit("pchipinterp", x, y)  
cinterp = fit("cubicinterp", x, y)  
sinterp = fit("splineinterp", x, y)  
xdata =  
vcat(minimum(x) :0.01:maximum(x),ma  
ximum(x))  
ydata = fm.(xdata)  
ydata1 = linterp.(xdata)  
ydata2 = ninterp.(xdata)  
ydata3 = pinterp.(xdata)  
ydata4 = cinterp.(xdata)  
ydata5 = sinterp.(xdata)  
figure("各插值方法的对比")  
scatter(x, y)  
hold("on")  
plot(xdata,ydata,"k:")  
plot(xdata,ydata1)  
plot(xdata,ydata2)  
plot(xdata,ydata3)  
plot(xdata,ydata4)  
plot(xdata,ydata5)  
legend(["原数据点", "原函数", "线性", "  
最邻近", "pchip", "cubic", "spline"])
```



各插值方法的对比

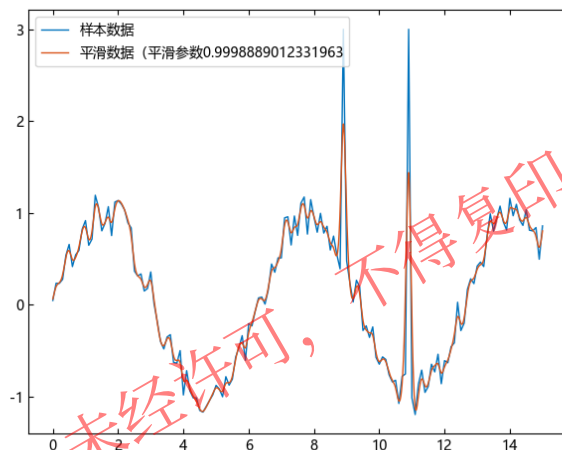
可以看出，cubic 方法与 pchip 方法实际上返回了一样的结果。

5.2 曲线曲面的平滑以及插值 – 对曲线数据进行平滑

假设我们有数据集 xdata, ydata, 首先使用 fit 函数的 smoothingspline 方法对它进行平滑。

```
xdata = 0:0.1:15
rng = MersenneTwister(1234)
ydata = sin.(xdata) + 0.5 .* (rand(rng,
length(xdata)) .- 0.5)
ydata[[90, 110]] .= 3

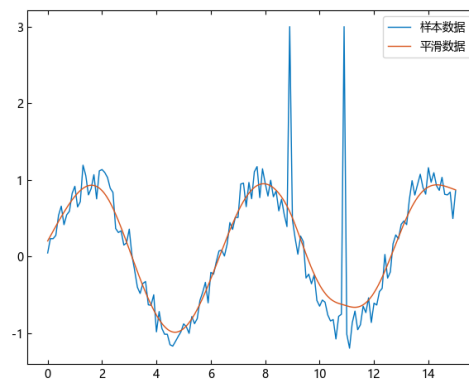
xsdata = 0:0.01:15
figure("平滑样条方法")
ft = fit("smoothingspline", xdata, ydata)
plot(xdata, ydata)
hold("on")
plot(xsdata, ft.(xsdata))
legend(["样本数据", "平滑数据 (平滑参数
$(ft.options.smoothingparams)"])
```



平滑样条方法

该方法返回的平滑参数很接近于 1, 导致这样的平滑过程过度了, 指定平滑参数为 0.6, 再次进行平滑。

```
figure("调整的平滑样条方法")
ft2 = fit("smoothingspline", xdata,
ydata, smoothingparams=0.6)
plot(xdata, ydata)
hold("on")
plot(xsdata, ft2.(xsdata))
legend(["样本数据", "平滑数据"])
```



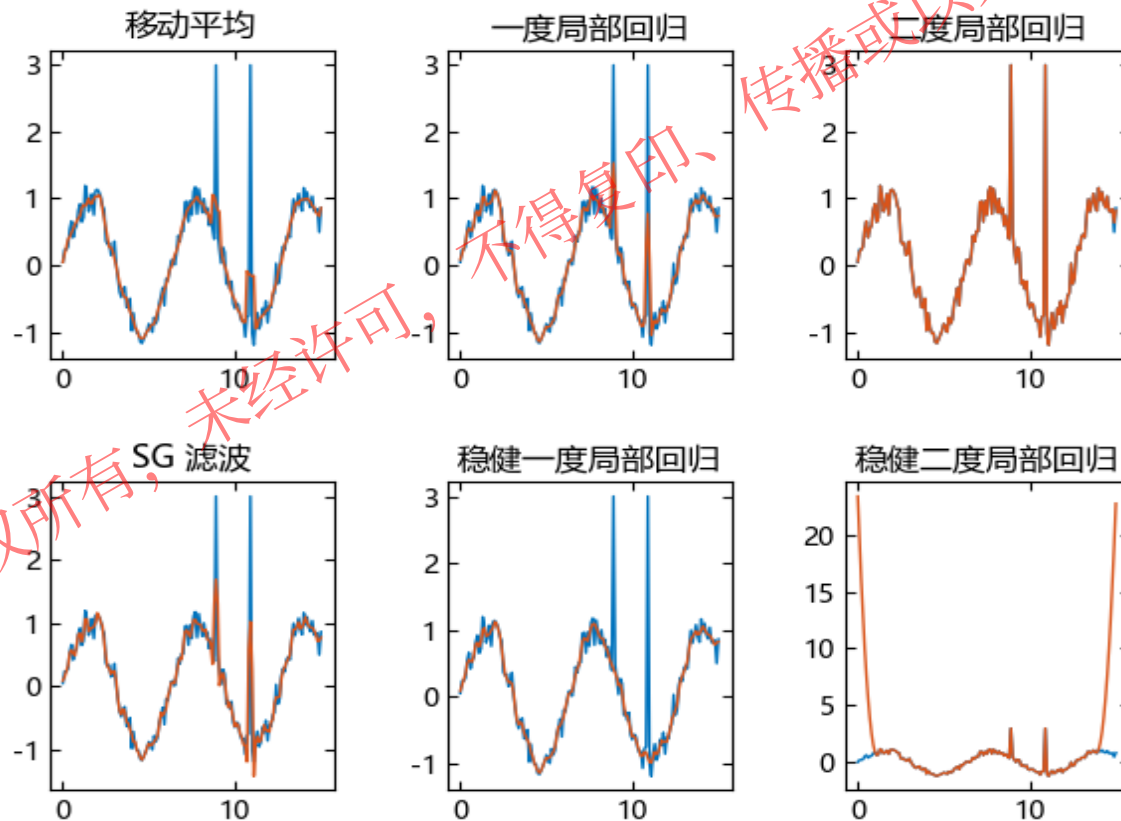
调整的平滑样条方法

可以看出, 在手动调整算法使用的平滑参数后, 平滑数据的结果就更有逻辑了。

5.2 曲线曲面的平滑以及插值 – 对曲线数据进行平滑

将 fit 平滑的结果与 smooth 各方法的平滑结果进行对比。

```
figure("各平滑方法的比较")
ydata_mov = smooth(xdata, ydata, "moving")
ydata_low = smooth(xdata, ydata, "lowess")
ydata_loe = smooth(xdata, ydata, "loess")
ydata_sgo = smooth(xdata, ydata, "sgolay")
ydata_rlow = smooth(xdata, ydata, "rlowess")
ydata_rloe = smooth(xdata, ydata, "rloess")
subplot(2, 3, 1)
title("移动平均")
hold("on")
plot(xdata, ydata)
plot(xdata, ydata_mov)
subplot(2, 3, 2)
title("一度局部回归")
hold("on")
plot(xdata, ydata)
plot(xdata, ydata_low)
subplot(2, 3, 3)
title("二度局部回归")
hold("on")
plot(xdata, ydata)
plot(xdata, ydata_loe)
subplot(2, 3, 4)
title("SG 滤波")
hold("on")
plot(xdata, ydata)
plot(xdata, ydata_sgo)
subplot(2, 3, 5)
title("稳健一度局部回归")
hold("on")
plot(xdata, ydata)
plot(xdata, ydata_rlow)
subplot(2, 3, 6)
title("稳健二度局部回归")
hold("on")
plot(xdata, ydata)
plot(xdata, ydata_rloe)
tightlayout()
```



平滑方法的对比

5.3 曲线曲面的平滑以及插值 – 对曲面数据进行平滑

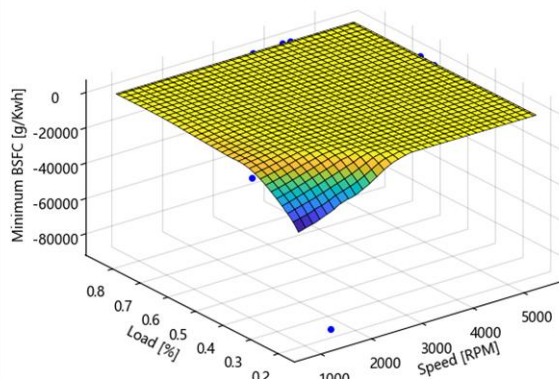
加载和预处理数据

```
include(pkgdir(TyCurveFitting) * "/examples/docs/Engine_Data_SI_NA_2L_I4.jl")
SL = ty_unique([SPEED LOAD_CMD], byrows=true)
nRuns = size(SL, 1)
minBSFC = zeros(nRuns)
Load = zeros(nRuns)
Speed = zeros(nRuns)
for i = 1:nRuns
    idx = @. (SPEED == SL[i, 1]) & (LOAD_CMD == SL[i, 2])
    minBSFC[i] = minimum(BSFC[idx])
    Load[i] = mean(LOAD[idx])
    Speed[i] = mean(SPEED[idx])
end
```

对预处理数据进行局部二次回归平滑处理，并将其绘制出来

```
f1 = fit("lowess", [Speed Load], vec(minBSFC), normalize=true)
figure("预拟合")
plot3fit(f1, [Speed Load], minBSFC)
xlabel("Speed [RPM]")
ylabel("Load [%]")
zlabel("Minimum BSFC [g/Kwh]")
```

我们发现原数据存在小于 0 的点，这并不符合问题的实际背景，因此我们将这些点剔除并重新进行拟合。

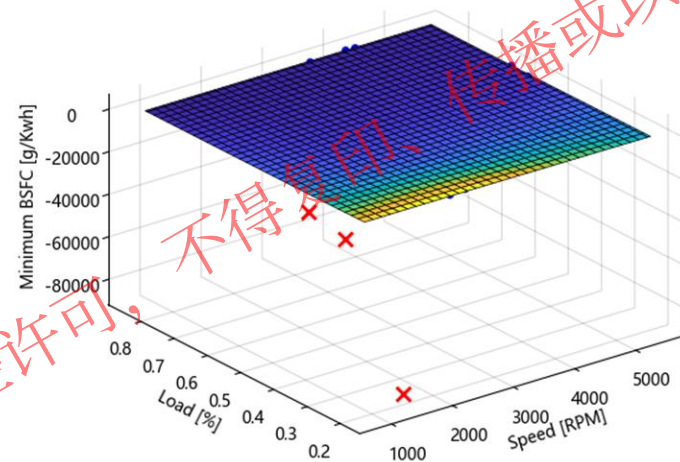


预拟合

5.3 曲线曲面的平滑以及插值 – 对曲面数据进行平滑

将负数据排除，并重新进行和绘制拟合，排除点用红色 x 号表示。

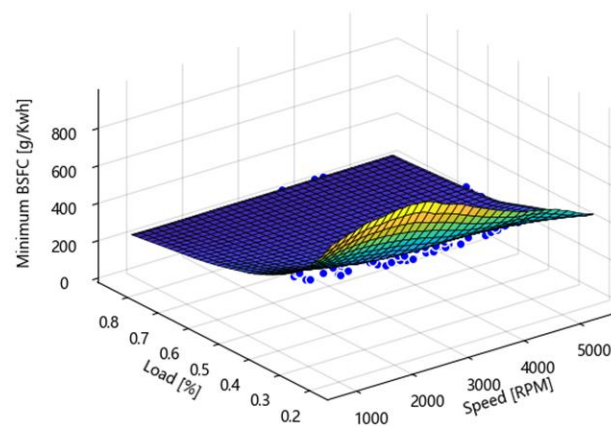
```
out = excludedata(Speed, minBSFC, "range", [0,
Inf])
f2 = fit("lowess", [Speed Load], minBSFC,
normalize=true, exclude=out)
figure("排除负数据值")
plot3fit(f2, [Speed Load], minBSFC, "exclude",
out)
xlabel("Speed [RPM]")
ylabel("Load [%]")
zlabel("Minimum BSFC [g/Kwh]")
```



排除负数据值

我们只关心最后数据值取正值的那部分函数图形，因此将图形缩进至我们感兴趣的部分。

```
zlim([0, maximum(minBSFC)])
```



排除负数据值后
感兴趣的部分

5.3 曲线曲面的平滑以及插值 – 对曲面数据进行平滑

我们希望知道拟合后的函数什么时候取最小，因此我们使用 plot3fit 函数绘制该函数的等高线图。

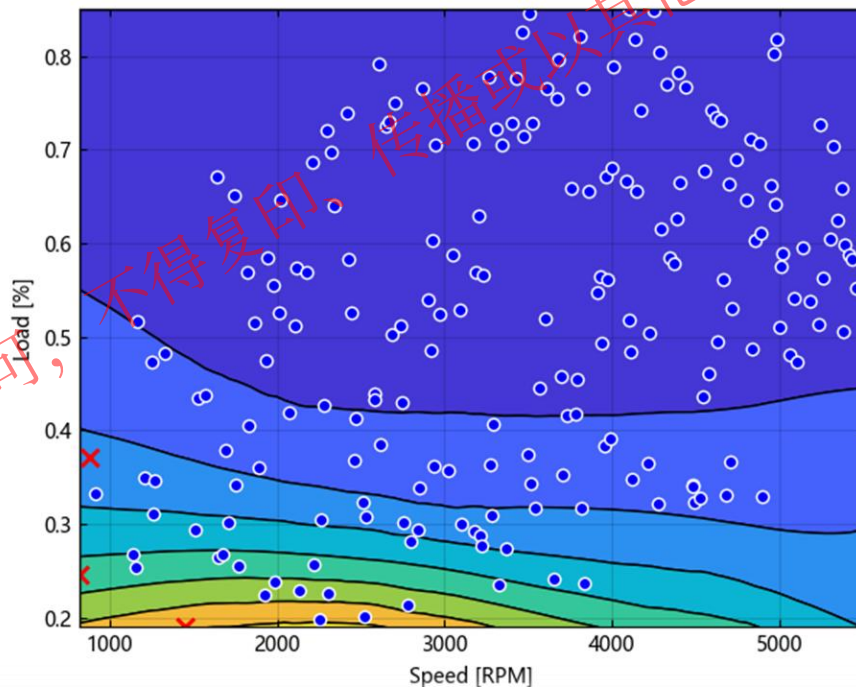
```
plot3fit(f2,[Speed Load],minBSFC,"exclude",out,"style","contour")
xlabel("Speed [RPM]")
ylabel("Load [%]")
```

拟合完成后，我们可以将拟合结果作为函数直接进行求值和调用。

```
speedbreakpoints = LinRange(1000, 5500, 17)
loadbreakpoints = LinRange(0.2, 0.8, 13)
tSpeed, tLoad = meshgrid2(speedbreakpoints, loadbreakpoints)
tBSFC = f2(tSpeed, tLoad)
tBSFC[1:2:end, 1:2:end]
```

7×9 Matrix{Float64}:

#	722.328	766.761	779.43	757.457	694.538	624.41	576.524	532.153	466.961
#	503.988	499.92	481.724	458.28	427.734	422.11	412.162	396.321	398.02
#	394.758	364.342	336.181	330.155	329.164	328.181	329.114	335.387	346.388
#	333.774	307.774	295.178	291.207	290.364	290.017	287.867	286.308	291.008
#	295.973	282.757	273.829	270.887	269.848	271.055	270.55	269.684	272.205
#	273.751	264.517	259.763	257.922	256.935	258.323	258.664	258.03	260.527
#	251.565	247.675	247.275	247.47	247.357	248.243	248.814	249.008	250.417



函数的等高线图

目录

1. 曲线拟合功能概述
2. 曲线拟合的数学模型
3. 曲线拟合及其实例
4. 曲面拟合及其实例
5. 曲线曲面的平滑以及插值
6. 关于样条及其构造
7. 样条的拟合平滑以及插值

6.1 关于样条及其构造- 样条的介绍

样条为在节点相交的多项式序列。样条在使用单一逼近多项式不实际的场景很有用。

曲线拟合工具箱函数允许您为拟合以及平滑数据构造样条。样条可以用来平滑含噪数据以及进行插值。

使用 `fit` 函数，您可以拟合三次样条插值、平滑样条以及薄板样条。其他的曲线拟合工具箱函数支持更特化的对于样条构造的控制。

可以使用 `spterm`s 函数查看一些术语的相关描述，所有支持的描述可以使用 `spterm`s() 查看。

```
spterm("B-form")
```

```
#= B-form: B-型。
```

```
B-型样条提供阶数、节点序列，以及在此指定阶和节点序列下表示  
该样条的B样条加权系数。=#
```

```
spterm("basis function")
```

```
#= basis function: 基函数。
```

```
st型样条的基函数使基于该类型样条中心点平移以提供加权和各项  
的函数，该加权和即为该类型样条所描述的函数。=#
```

6.1 关于样条及其构造- 样条的介绍

构造函数	类型	单元 or 多元	结构体名称	组成要素	描述
ppmak	pp 型	均支持	PPMAK	断点、系数、维度	分段多项式样条
rpmak	rp 型 (rs 型)	均支持	PPMAK		分段多项式组成的有理样条
rsmak	rb 型 (rs 型)	均支持	SPMAK	节点、系数、维度	B 样条组成的有理样条
spmak	b 型	均支持	SPMAK		B 样条
stmak	tp00 (st 型)	多元	STMAK	中心、维度、维度、基本区间	薄板样条
	tp10 (st 型)	多元			薄板样条的一阶导
	tp01 (st 型)	多元			薄板样条二阶导
	tp (st 型)	多元			默认 st 型

6.2 关于样条及其构造 - 分段多项式样条

域元	描述	说明
breaks	断点向量	严格增的向量，长度为 $l+1$
coefs	系数矩阵	大小为 (l,k) ， k 为多项式阶数。 $coefs[j,:]$ 为第 j 个多项式的 k 个系数。
l	多项式片段数	
k	多项式阶数	
d	样条维度	对于单变元样条，该值为 1

- **基本区间**: 多项式直接定义的区域，即 $[breaks[1], breaks[l])$ ，这也是使用 `fnplt` 绘制的默认区间。
- **公式**: 基本区间以内，对 $breaks[j] \leq t < breaks[j+1]$
$$f(t) = polyval(coefs[j,:], t - breaks[j])$$
- 这里 `polyval(a,x)` 为基础数学库函数，它将会返回值 $\sum_{j=1}^k a[j]x^{k-j}$ 。
- 基本区间以外为第一（最后一个）多项式片段的延伸。
- **连续性**: 函数值和（或）其导数值可能在其内部断点 $breaks[2:l]$ 上存在阶跃。

6.2 关于样条及其构造- 分段多项式样条

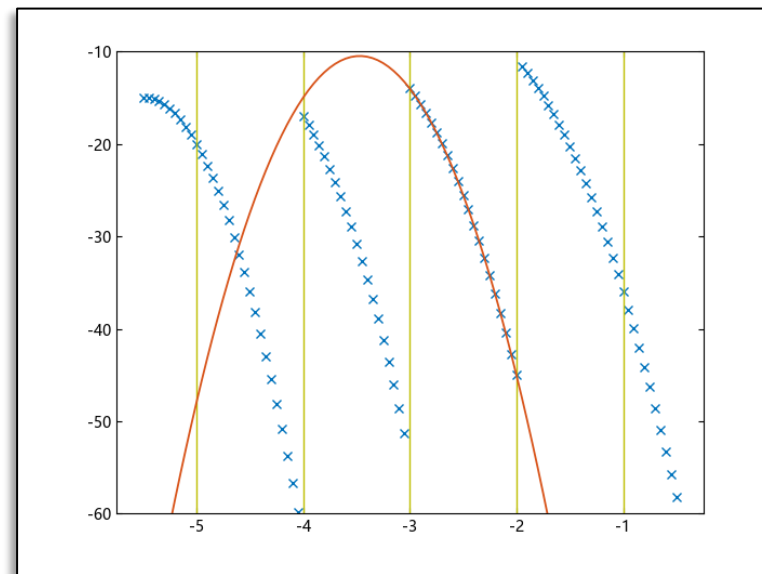
- 构造具有断点序列 -5:-1 以及系数序列 -22:-11 的分段多项式，因为这个断点序列有 5 个元素，因此有 4 个分段区间，而系数序列有 12 个元素，因此您实际上指定了一个 3 ($12/4=3$) 阶多项式。
- 构造基本图，使用 fnval 进行样条函数求值。
- 为绘图添加断点线，使用 fnbrk 获得样条的断点坐标以及断点个数。
- 把第三个多项式片段的多项式叠加在上面。使用 fnbrk 获得样条特定的多项式片段。
- 其中蓝色的 x 符号为样条求值，黄色的竖线为断点线，红色的曲线为样条的第三个多项式片段。

```
pp = ppmak(-5:-1,-22:-11)

x = LinRange(-5.5,-.5,101)
figure("PPMAK")
plot(x,fnval(pp,x),"x")

breaks = fnbrk(pp,"b")
yy = [-60,-10]
hold("on")
for j = 1:fnbrk(pp,"l")+1
    plot(breaks[[j,j]],yy,"y")
end

plot(x,fnval(fnbrk(pp,3),x),linewidth=1.3)
ylim([-60 -10])
hold("off")
```



PPMAK

6.3 关于样条及其构造- B 样条

域元	描述	说明
t(knots)	节点向量	非递减的向量，长度为 $n+k$
coefs	系数矩阵	大小为 (d,n) , $coefs[j,:]$ 为第 j 个变元对应的 B 样条系数。
n	控制点个数	
k	多项式阶数	
d	样条维度	对于单变元样条，该值为 1

- **基本区间**: 多项式直接定义的区域, 即 $[t[1],t[n+k]]$, 这也是使用 `fnplt` 绘制的默认区间。
- **公式**: 基本区间以内, $f = \sum_{i=1}^n B_{i,k} a[:,i]$
- 其中 $B_{i,k} = B(\cdot | t[i:i+k])$ 为具有给定节点序列 t , 即 $t[i],...,t[i+k]$, 阶数为 k 的第 i 个 B 样条。基本区间以外样条未定义。
- **连续性**: 函数值和 (或) 其导数值可能在其内部断点节点 $t[i]$, 其中 $t[1] < t[i] < t[n+k]$ 以及其端点节点 $t[1]$ 与 $t[n+k]$ 上存在阶跃。

6.3 关于样条及其构造 - B 样条

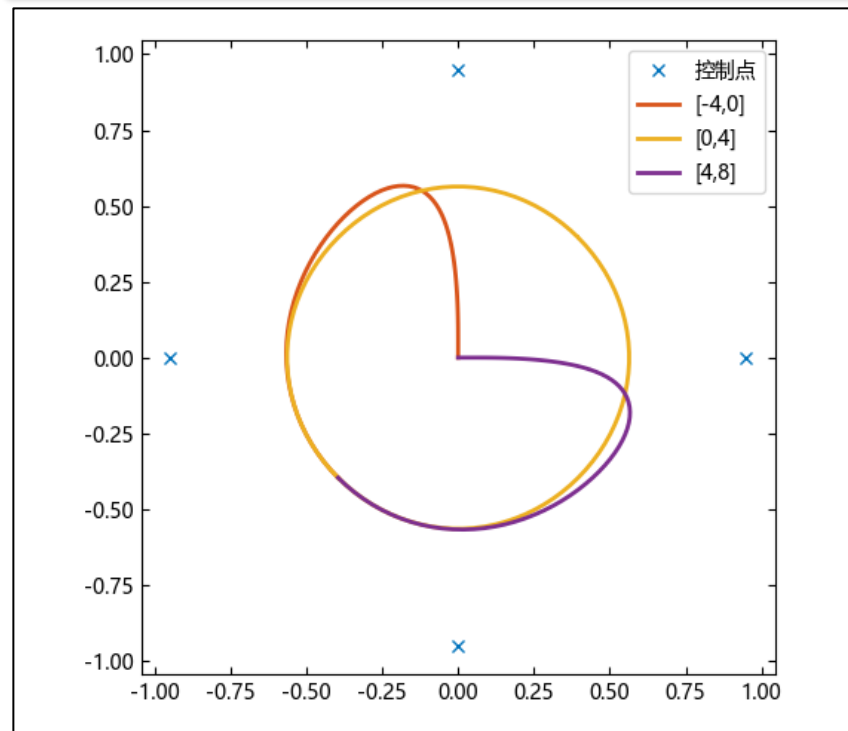
- 构造一个具有均匀节点序列和 8 个控制点的 B 样条。
- 该样条的参数范围是从 -4 到 8，分别从 [-4,0],[0,4],[4,8] 参数范围使用 `fnplt` 绘制该样条，并同时绘制其控制点。
- 从图形可以发现，该样条在 [0,4] 的部分很接近一个圆，因此我们通过 `fnder` 和 `fnval` 函数求该样条的微分来计算该样条在 21 个点的曲率。
- 虽然这个曲率不完全是常数，但是它很接近圆半径的倒数，亦即为

```
t = LinRange(0, 4, 21)
zt = zeros(size(t))
dsp = fnder(sp)
dspt = fnval(dsp, t)
ddspt = fnval(fnder(dsp), t)
kappa = abs.(dspt[1, :] .* ddspt[2, :] - dspt[2, :] .*
ddspt[1, :]) ./ (sum(dspt.^2, dims=1)'.^(3 / 2))
(minimum(kappa), maximum(kappa))
## (1.6747265870207704, 1.8610852567389633)
```

```
1 / norm(fnval(sp, 0))
## 1.7863750261554892
```

```
points = 0.95 * [0 -1 0 1; 1 0 -1 0]
sp = spmak(-4:8, [points points])
```

```
figure("SPMAK")
plot(points[1, :], points[2, :], "x")
hold("on")
fnplt(sp, [-4, 0])
fnplt(sp, [0, 4])
fnplt(sp, [4, 8])
axis("square")
legend(["控制点", "[-4,0]", "[0,4]", "[4,8]"])
hold("off")
```



SPMAK

6.4 关于样条及其构造 - 有理样条与 NURBs

有理样条定义为两个样条的比 $r(x) = s(x)/w(x)$ ，这要求 w 为标量值样条， s 常常为向量值样条，而且我们通常希望 $w(x)$ 在我们感兴趣的点非零。有理样条相比于通常的样条可以用来准确地描述一些基本形状，例如圆锥。

有理样条的定义中并没有涉及 s 与 w 的相关性，但是在目前的工具箱中，要求这两个样条不仅是同样的类型，也需要具有相同的阶数、断点（或结点）。这种情况下，就可以用 $R(x) = [s(x); w(x)]$ 来描述这样的有理样条。

这种样条的值向量为样条 r 的值向量再加一个元素，因此这称之为 rs 型有理样条。更细致的说，根据 s 和 w 是 pp 型还是 B 型，可以把 rs 型再分为 rp 型和 rB 型。

使用普通样条表示有理样条的方式使得在曲线拟合工具箱中对有理样条使用 fn 函数变得容易了很多，但依然存在以下的例外。有理样条的积分不一定还是有理样条，因此 $fnint$ 不可能推广到有理样条。有理样条的微分还是有理样条，但是阶数一般翻了个倍，因此， $fnder$ 和 $fndir$ 也不能对有理样条使用。但是，目前有个指令 $fntlr$ 可以用来计算给定函数在给定点一直到给定阶数的微分值。

NURBs（非均匀有理 B 样条）是一种特殊的有理样条，定义为 s 和 w 均为 B 型，且 s 中的每个系数都包含 w 的对应系数作为因子的有理样条，即 $s = \sum_i B_i v[i] a[:, i] = \sum_i B_i v[i]$ 。 rB 型的分子样条的正规系数 $a[:, i]$ 相比于其非正规化系数 $v[i] a[:, i]$ 更常被用来作为控制点。

工具箱并没有特殊的提供 NURBs 型，而是提供了更通常的有理样条类型， rB 型与 rp 型。

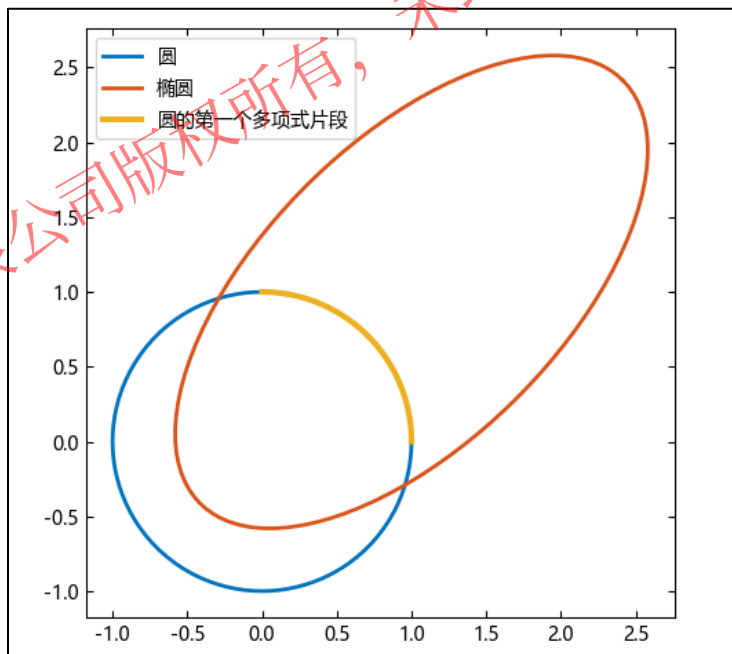
6.4 关于样条及其构造 - 有理样条与 NURBs

- 使用 `rsmak` 生成一个具有标准几何形状有理样条并绘制该样条。
- 该样条是个圆。使用 `fncmb` 将其伸缩变成一个椭圆，使用 `fncmb` 将其作用于旋转矩阵将其旋转 45 度，再使用 `fncmb` 将其向右和向上平移一个坐标，将这些作用之后的图形盖在圆上。
- 这个圆实际上是由四个片段组成，为了显示其第一个片段，首先使用 `fn2fm` 将该样条转为 `rp` 型，再使用 `fnbrk` 取出其第一个多项式片段，最后再将该图形绘制在之前的图上。为了突出显示，将其线宽设置为 3。

```
circle = rsmak("circle")
figure("有理样条")
fnplt(circle)

ellipse = fncmb(circle, [2 0; 0 1])
s45 = 1 / sqrt(2)
rtellipse = fncmb(fncmb(ellipse, [s45 -s45; s45 s45]),
[1; 1])
hold("on")
fnplt(rtellipse)
axis("square")

quarter = fnbrk(fn2fm(circle, "rp"), 1)
fnplt(quarter, 3)
legend(["圆", "椭圆", "圆的第一个多项式片段"])
hold("off")
```



有理样条

6.5 关于样条及其构造 - st 样条

由张量积构造的多变元函数称之为分散变换型 (scattered translates form), 简称为 st 型。它使用一个固定函数 Ψ 或分散变换 $\Psi(\cdot - c_j)$ 以及一些多项式项, 具体地说

$$f(x) = \sum_{j=1}^{n-k} \Psi(x - c_j) a_j + p(x)$$

其中 Ψ 称之为基函数, 位点序列 c 称之为中心, 这对应了 n 个系数 a_j , 剩余的 k 个系数表示它的多项式 p 。

当基函数径向对称, 即 $\Psi(x)$ 仅与其输入 x 的欧式距离 $|x|$ 有关, 则 Ψ 称之为径向基函数, 对应的 f 称之为 RBF。

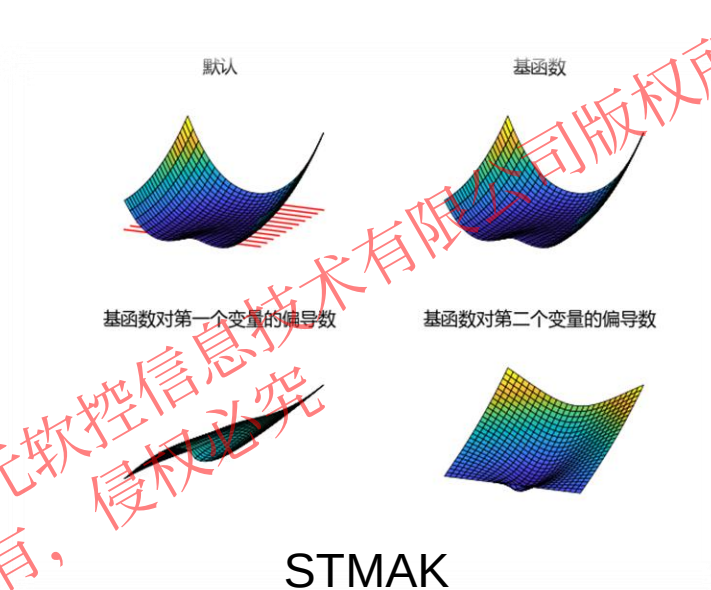
st 型的函数 f 线性地依赖其系数。假设您想要决定这些系数 a_j 使得函数 f 在给定点 x_i 等于给定值, 则需要 $(\Psi_j(x_i))$ 的配置矩阵, 您可以使用 `stcol(centers,x,type)` 来得到这样的矩阵。

st 型引人注意是因为与分段多项式类型不同, 它的复杂度对于任何数量的变量下都相同。这看上去很简单, 但由于其对于中心选择的完全自由, 又很灵活。

不好的是, 即使是对于径向基函数的最好选择, 计算任何位点时都会包含它所有系数的信息。比如, 使用 `fnplt` 绘制标量值薄板样条包含了 51×51 格点的值的计算, 如果这有大于或等于 1000 个系数就非常不容易。如果您想要决定这 1000 个系数使之在给定的 1000 个数据点等于给定值时, 情况就更糟了, 因为这包含求解一个稠密的 1000 阶线性方程组, 如果使用直接方法, 这需要 $O(10^9)$ 次浮点运算。仅仅是这个线性系统的配置矩阵的构造 (由 `stcol` 完成) 都需要 $O(10^6)$ 次。

6.5 关于样条及其构造 - st 样条

- 生成中心在原点，系数为 1 的 st 样条，这样生成的样条实际上表示薄板样条基函数 $\psi(x) = |x|^2 \log|x|^2$ ，注意，这里的 $|x|^2$ 表示向量 x 的欧氏距离。将该样条介于 $-1.5 \leq x \leq 1.5, 0 \leq y \leq 1.2$ 的图形绘制出来。
- 这个函数在原点附近取负值，绘制额外的直线表征 z 平面。
- 重新生成具有同样的中心和系数，但是类型分别为 “tp00”，“tp10” 和 “tp01” 的样条，这分别表示原样条以及其对于第一个和原样条对于第二个系数的偏导数。把他们在同样的范围里的图绘制出来。



```
inx = [-1.5 1.5]
iny = [0 1.2]
inz = [0 0]
figure("STMAK")
subplot(2, 2, 1)
fnplt(stmak([0; 0], 1), [inx, iny])
```

```
hold("on")
plot3(inx[:, repeat(LinRange(iny[1], iny[2], 11)', 2,
1), inz[:, color="r")
plt_view(25, 20)
axis("off")
title("默认")
```

```
subplot(2, 2, 2)
fnplt(stmak([0; 0], [1 0 0 0], "tp00", (inx, iny)))
hold("on")
title("基函数")
plt_view(25, 20)
axis("off")
subplot(2, 2, 3)
fnplt(stmak([0; 0], [1 0], "tp10", (inx, iny)))
hold("on")
title("基函数对第一个变量的偏导数")
plt_view(25, 20)
axis("off")
subplot(2, 2, 4)
fnplt(stmak([0; 0], [1 0], "tp01", (inx, iny)))
hold("on")
title("基函数对第二个变量的偏导数")
plt_view(25, 20)
axis("off")
```

6.6 关于样条及其构造- 多元样条与薄板平滑样条

多元样条可以由单元样条使用张量积构造生成, 比如, 一个三元 B 样条可以表示为

$$f(x, y, z) = \sum_{u=1}^U \sum_{v=1}^V \sum_{w=1}^W B_{u,k}(x) B_{v,l}(y) B_{w,m}(z) a_{u,v,w}$$

其中 $B_{u,k}$, $B_{v,l}$, $B_{w,m}$ 为单元 B 样条。类似地, 张量积样条的 pp 型是由各系数上的断点序列以及每个由此指定的超长方体的系数数组所指定的。

一个非常不同的二元样条为薄板样条, 这也是目前工具箱中唯一的 st 型样条, 它为具有以下形式的函数

$$f(x) = \sum_{j=1}^{n-3} \Psi(x - c_j) a_j + x[1] a_{n-2} + x[2] a_{n-1} + a_n$$

其中 $\Psi(x) = |x|^2 \log|x|^2$ 为薄板样条基函数, $|x|$ 表示向量 x 的欧式距离。

薄板样条为双元平滑样条, 即薄板样条在所有足够光滑的函数 f 中最小化

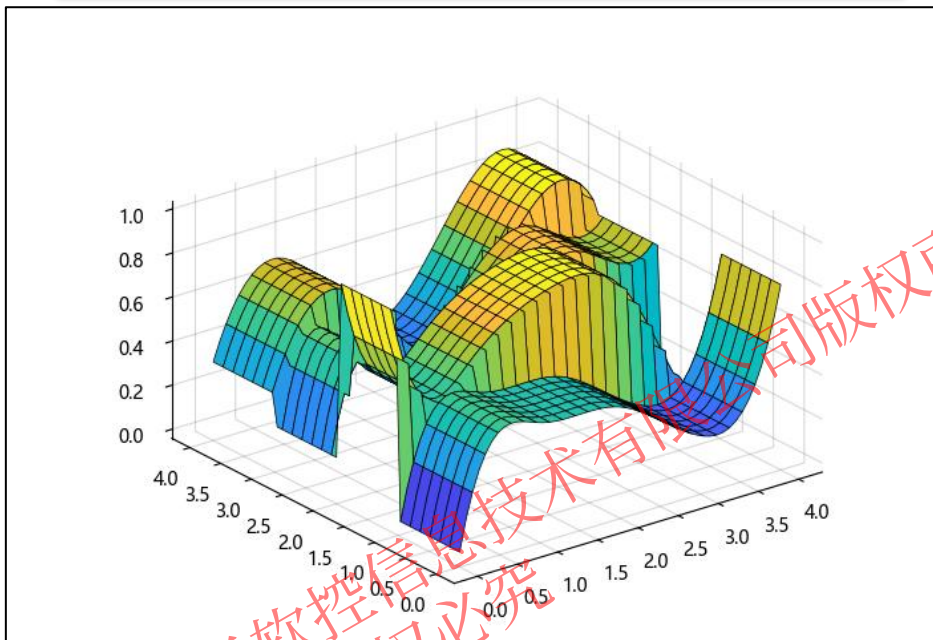
$$p \sum_{i=1}^{n-3} |y_i - f c_i^2| + (1 - p) \int (|D_1 D_1 f|^2 + 2 |D_1 D_2 f|^2 + |D_2 D_2 f|^2)$$

这里, y_i 为数据位点 c_i 的数据值, p 为平滑参数, $D_j f$ 表示 f 关于 $x[j]$ 的偏导数。积分是在整个 $\mathbb{R}^2$ 中取的。上求和极限 $n-3$ 表示薄板样条的三个自由度在其多项式片段上。

6.6 关于样条及其构造- 多元样条与薄板平滑样条

生成一个第一个变量为三次（有 11 个节点和 7 个系数），但在第二个变量为分段常数（ $(2+5+2)-8=1$ ）的多元 pp 样条，并使用 fnplt 绘制该样条的图形。

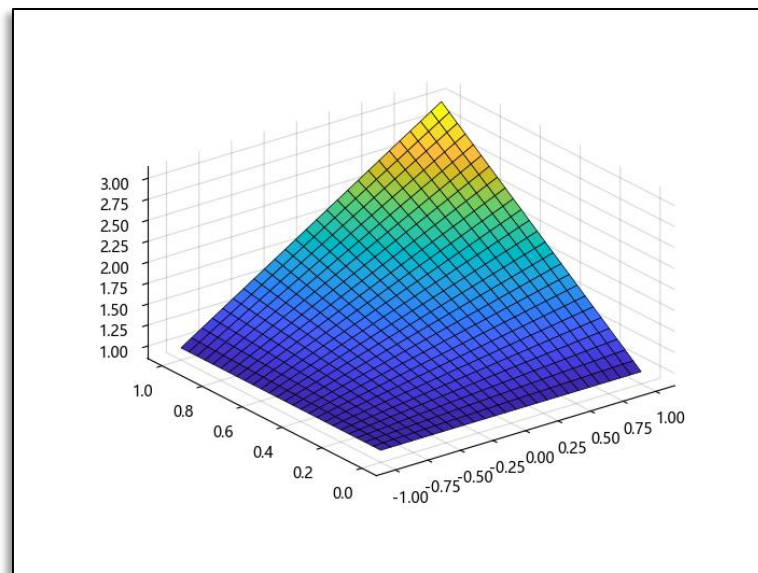
```
rng = MT19937ar(1)
figure("多元 B 样条")
fnplt(spmak((augknt([0:4...], 4)[1]', augknt([0:4...],
3)[1]'), rand(rng, 7, 8)))
```



多元 B 样条

生成一个定义在矩阵 $[-1,1] \times [0,1]$ 上第一个变量为线性，第二个变量为常数的多项式，并使用 fnplt 绘制该多项式的图形。

```
coefs = [1 0; 0 1]
pp = ppmak([-1 1], [0 1]), coefs)
figure("多元 pp 样条")
fnplt(pp)
```



多元 pp 样条

目录

1. 曲线拟合功能概述

2. 曲线拟合的数学模型

3. 曲线拟合及其实例

4. 曲面拟合及其实例

5. 曲线曲面的平滑以及插值

6. 关于样条及其构造

7. 样条的拟合平滑以及插值

7.1 样条的拟合平滑及其插值- 介绍

目前曲线拟合工具箱拥有的使用样条进行拟合、平滑及插值的函数如下

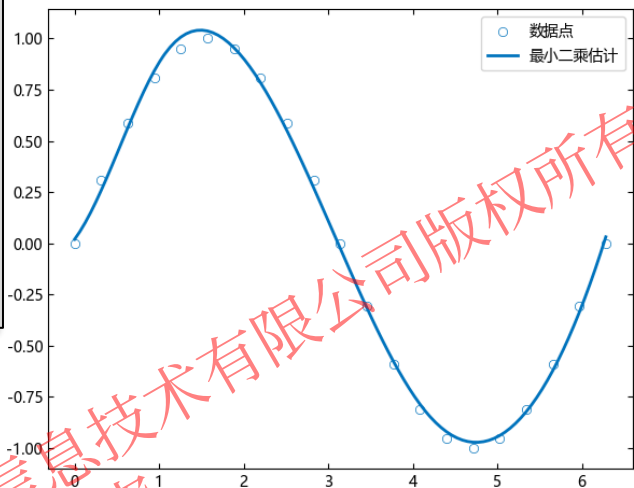
类别	名称	描述	使用的样条类
插值	csape	带端点条件的三次样条插值	PPMAK: pp
插值	csapi	三次样条插值	PPMAK: pp
平滑	csaps	三次平滑样条	PPMAK: pp
插值	cscvn	“自然”或周期性插值三次样条曲线	PPMAK: pp
插值	rscvn	分段双圆弧 Hermite 插值	SPMAK: rb
拟合	spap2	最小二乘样条估计	SPMAK: b
插值	spapi	样条插值	SPMAK: b
平滑	spaps	平滑样条	SPMAK: b
平滑	tpaps	薄板平滑样条	STMAK: st

7.2 样条的拟合平滑及其插值- 样条拟合 (spap2)

`spline = spap2(knots, k, x, y)` 返回给定节点序列 `knots`, 阶数为 `k` 的 B 样条 `f`, 满足 $y[:,j] = f(x[j]), \forall j$, 加权均方的意义下极小化下和 $\sum_j w[j] |y[:,j] - f(x[j])|^2$, 其中权默认为 1。数据值 `y[:,j]` 可以为标量、向量、矩阵或者数组, $|z|^2$ 为 `z` 的所有元素的平方和。同一位点的数据点由其均值代替。

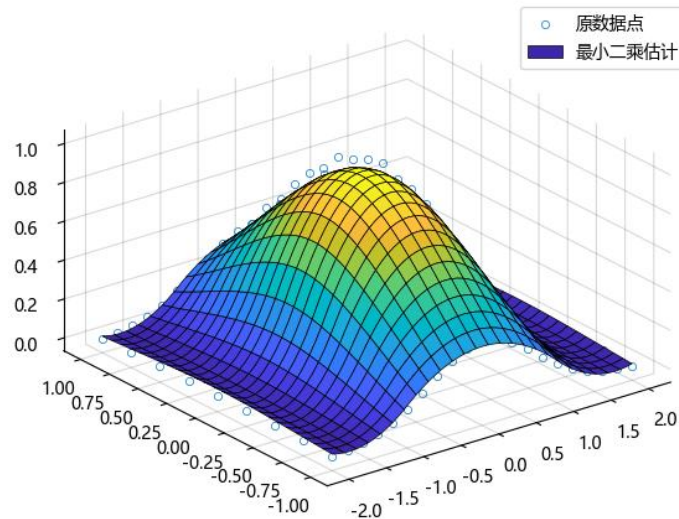
如果位点 `x` 满足 Schönberg-Whitney 条件 $knots[j] < x[j] < knots[j + k]$ 其中 $j = 1:length(x) = length(knots) - k$, 则存在给定阶数与节点序列的样条满足第一个公式。如果 `x` 不存在子序列满足上式, 也不会有任何样条得到返回。

```
x = (0:0.05:1).*(2*pi)
y = sin.(x)
sp =
spap2(augknt([0,pi/3,4*pi/3,2*pi],4)[1],4,x,y)
figure("三次样条的最小二乘估计")
scatter(x,y)
hold("on")
fplot(sp)
legend(["数据点","最小二乘估计"])
```



三次样条的最小二乘估计

```
x = -2:.2:2
y = -1:.25:1
xx, yy = ndgrid(x, y)
z = exp(-(xx.^2+yy.^2))
figure("二元函数的最小二乘估计")
scatter3(xx,yy,z)
hold("on")
sp = spap2((augknt(-2:2,3)[1],2),
[3 4], (x, y), z)
fplot(sp)
legend(["原数据点","最小二乘估计"])
```



二元函数的最小二乘估计

7.3 样条的拟合平滑及其插值- 插值

csape – 带端点条件的三次样条插值，该函数可选择的端点条件如下， $pp = csape(x,[e1,y,e2],conds)$

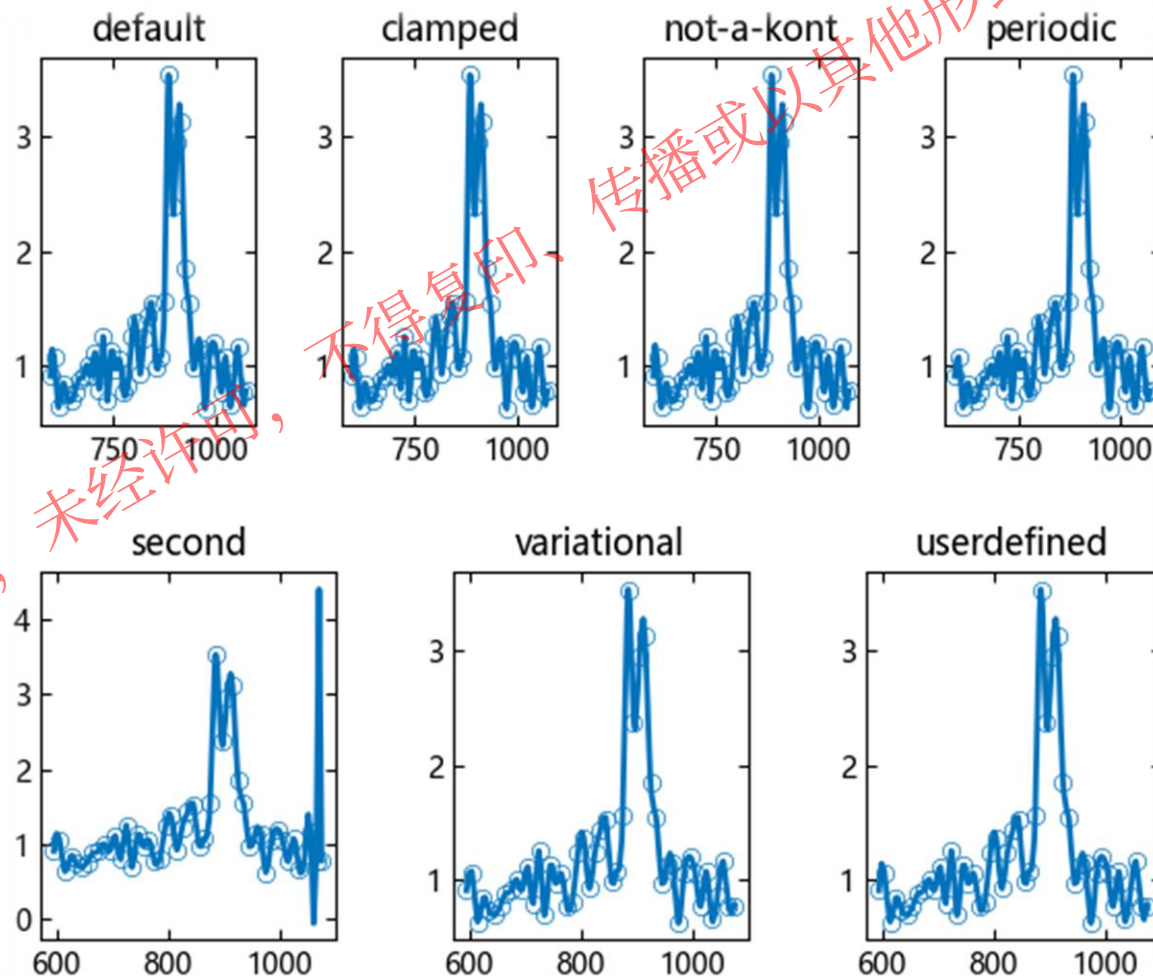
端点条件	含义
"clamped" 或 "complete"	将端点斜率与给定的值 $e1$ 和 $e2$ 匹配。如果不提供 $e1$ 和 $e2$ 的值，则此选项匹配默认的拉格朗日边界条件。
"not-a-knot"	忽视第二个和倒数第二个位点。此选项会忽略 $e1$ 和 $e2$ 被提供的任何值。
"periodic"	将左端的一阶和二阶导数与右端的一阶导数和二阶导数匹配。
"second"	将端点的二阶导数与给定的值 $e1$ 和 $e2$ 匹配。如果不提供 $e1$ 和 $e2$ 的值，则此选项对两者都使用默认值 0 。这种情况下，此选项与"variational"相同。
"variational"	将端点二阶导数设置为零。此选项会忽略 $e1$ 和 $e2$ 被提供的任何值。
1×2 矩阵，矩阵元素为 $0, 1, 2$	为每一端指定不同的端点条件，此矩阵元素的元素指定由边界条件固定的样条导数的阶数。



7.3 样条的拟合平滑及其插值- 插值

```
x, y = titanium()
rng = MT19937ar(1)
y = y.*(1 .+ rand(rng,1,length(y)))
s0 = csape(x, y)
s_clamped = csape(x, y, "clamped")
s_nak = csape(x, y, "not-a-kont")
s_per = csape(x, y, "periodic")
s_second = csape(x, hcat(fnval(fnder(fnbrk(s0, 1), 2), x[1])[1],
y, fnval(fnder(fnbrk(s0, s0.pieces-1), 2), x[2])[1]), "second")
s_var = csape(x, y, "variational")
s_user = csape(x, hcat(fnval(fnder(s0), x[1]), y, y[end]), [1, 0])
figure("csape 的边界条件对比")
subplot(2, 4, 1)
title("default")
hold("on")
scatter(x, y)
fnplt(s0)
subplot(2, 4, 2)
title("clamped")
hold("on")
scatter(x, y)
fnplt(s_clamped)
subplot(2, 4, 3)
title("not-a-kont")
hold("on")
scatter(x, y)
fnplt(s_nak)
subplot(2, 4, 4)
title("periodic")
hold("on")
scatter(x, y)
fnplt(s_per)
tightlayout()

subplot(2, 3, 4)
title("second")
hold("on")
scatter(x, y)
fnplt(s_second)
subplot(2, 3, 5)
title("variational")
hold("on")
scatter(x, y)
fnplt(s_var)
subplot(2, 3, 6)
title("userdefined")
hold("on")
scatter(x, y)
fnplt(s_user)
```

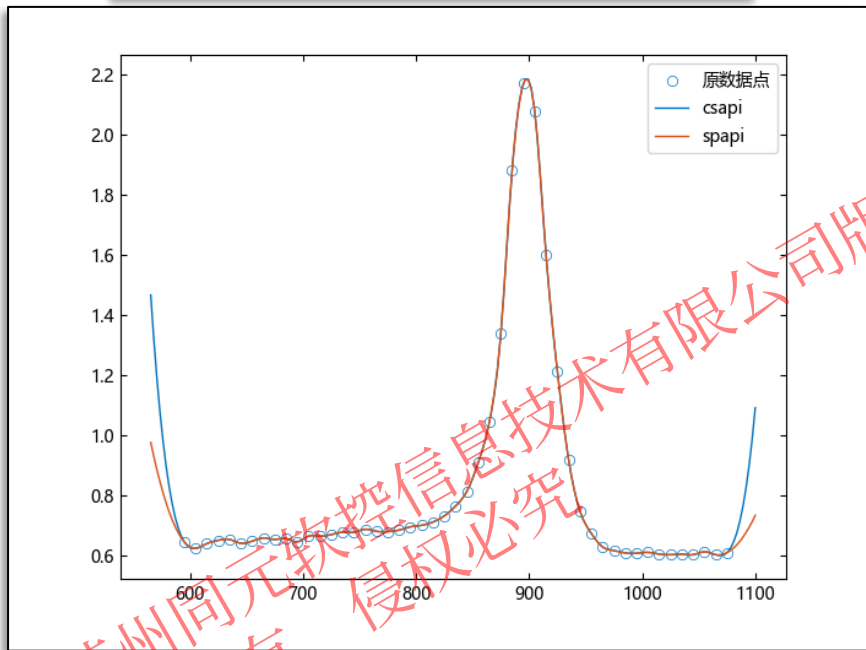


csape 插值的不同边界条件的对比

7.3 样条的拟合平滑及其插值- 插值

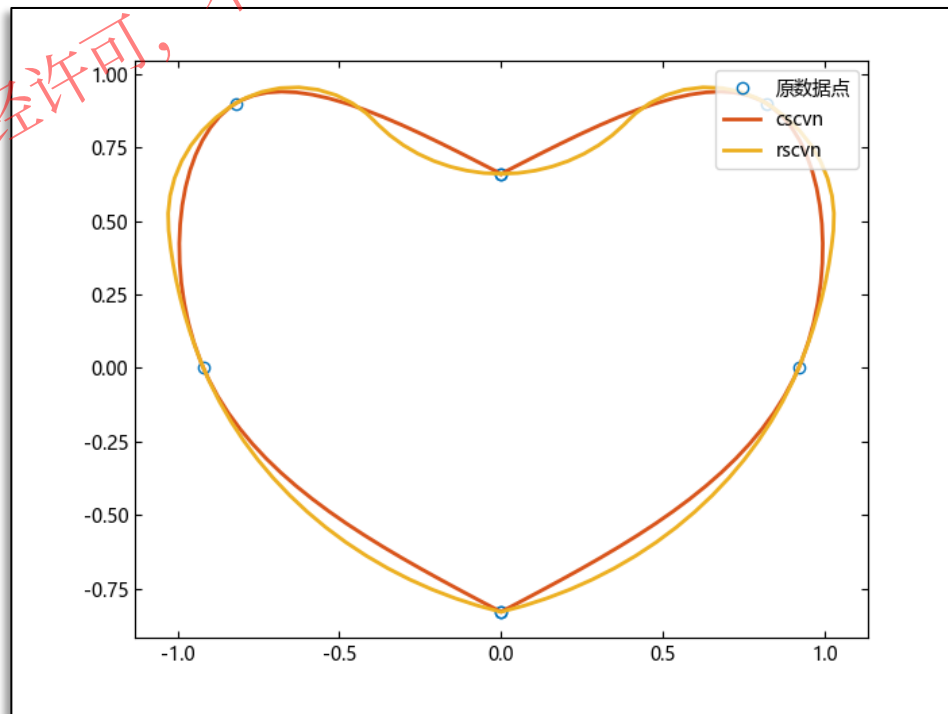
在剩余的四个插值方法中，`csapi` 和 `spapi` 为数据插值方法，`rscvn` 和 `cscvn` 为点插值方法（把平面点连成线）

```
x,y = titanium()
pp_csapi = csapi(x,y)
pp_spapi = spapi(3,x,y)
xdata = LinRange(560,1100,200)
yc = fnval(pp_csapi,xdata)[3:end]
ys = fnval(pp_spapi,xdata)[3:end]
figure("cspai 与 spapi 插值")
scatter(x,y)
hold("on")
plot(xdata[3:end],yc)
plot(xdata[3:end],ys)
legend(["原数据点", "csapi", "spapi"])
```



csapi 与 spapi 插值

```
points = [0 0.82 0.92 0 0 -0.92 -0.82 0; 0.66 0.9 0 -0.83  
-0.83 0 0.9 0.66]  
figure("cscvn 与 rscvn 插值")  
plot(points[1, :], points[2, :], "o")  
hold("on")  
fnplt(cscvn(points))  
fnplt(rscvn(points))  
legend(["原数据点", "cscvn", "rscvn"], loc="northeast")
```



cscvn 与 rscvn 插值

7.4 样条的拟合平滑及其插值- 平滑

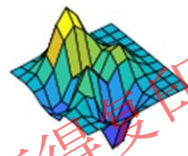
名称	用法	描述
csaps	csaps(x,y,p,xx,w)	平滑样条 f 最小化 $p \sum_{i=1}^n w_j y_j - f(x_j) ^2 + (1 - p) \int \lambda(t) D^2 f(t) ^2 dt$
spaps	spaps(x,y,tol,w,m)	定义曲线与点之间的距离为 $E(f) = \sum_{i=1}^n w_j y_j - f(x_j) ^2$ 定义粗糙测度为 $F(D^m f) = \int_{\text{minimum}(x)}^{\text{maximum}(x)} \lambda(t) D^m f(t) ^2 dt$ 当 tol 为非负值时, 样条 f 使 $\rho E(f) + F(D^m f)$ 最小
tpaps	tpaps(x,y,p)	定义粗糙测度为 $R(f) = \int (D_1 D_2 f ^2 + 2 D_1 D_2 f ^2 + D_2 D_2 f ^2)$ 这里的积分在整个 $\mathbb{R}^2$ 上取。则薄板平滑样条 f 为加权和 $pE(f) + (1 - p)R(f)$ 的唯一最小化子。

7.4 样条的拟合平滑及其插值- 平滑

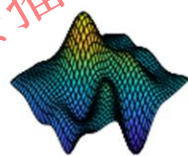
```
x = (LinRange(-2, 3, 11), LinRange(-3, 3, 11))
xx, yy = ndgrid(x[1], x[2])
plotx = (LinRange(-3, 3.5, 101), LinRange(-2, 3.5, 101))
plotxx, plotyy = ndgrid(plotx[1], plotx[1])
rng = MT19937ar(1)
y = peaks(xx, yy)
noisy = y .+ (rand(rng, size(y, 1), size(y, 2)) .- 0.5)
X1, X2 = meshgrid2(x[1], x[2])
figure("样条平滑函数对比")
subplot(2, 3, 1)
surf(xx, yy, y)
hold("on")
title("原始数据")
axis("off")
subplot(2, 3, 2)
sval, p = csaps(x, noisy, [], plotx)
surf(plotxx, plotyy, sval)
hold("on")
title("默认 csaps")
axis("off")
subplot(2, 3, 3)
ssval = csaps(x, noisy, 0.996, plotx)[1]
surf(plotxx, plotyy, ssval)
hold("on")
title("平滑参数为 0.996 的 csaps")
axis("off")
subplot(2, 3, 4)
surf(xx, yy, noisy)
hold("on")
title("含噪原始数据")
axis("off")
```

```
subplot(2, 3, 5)
st, = tpaps(hcat(vec(X1), vec(X2)), vec(noisy))
plotx1 = (LinRange(-2, 3, 101), LinRange(-3, 3, 101))
plotxx1, plotyy1 = ndgrid(plotx1[1], plotx1[2])
z_tpaps = fnval(st, plotx1)
surf(plotxx1, plotyy1, z_tpaps)
hold("on")
title("未外推的 tpaps")
axis("off")
subplot(2, 3, 6)
sp, = spaps(x, noisy, 0.1, [], 3)
z_spaps = fnval(fnxtr(sp, 3), plotx)
surf(plotxx, plotyy, z_spaps)
hold("on")
title("spaps")
axis("off")
tightlayout()
```

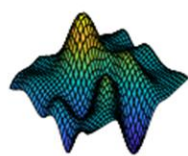
原始数据



默认 csaps



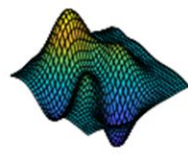
平滑参数为 0.996 的 csaps



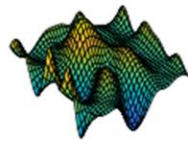
含噪原始数据



未外推的 tpaps



spaps



样条平滑函数对比

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听